

Complete Docker & Container Mastery Course for DevOps Engineering

From Absolute Beginner to Production-Level Containerization (Docker Engine on Ubuntu 24.04 LTS)

Prepared for Aryan

August 2026

Contents

1	How to Use This Course	8
2	PART 1 — Container Fundamentals	9
2.1	1.1 What Is a Process?	9
2.2	1.2 What Is Virtualization? What Is a VM?	9
2.3	1.3 What Is a Container?	9
2.4	1.4 VM vs Container	9
2.5	1.5 Container vs Application, Container vs Image, Container vs Docker	10
2.6	1.6 Why Containers Were Created	10
2.7	1.7 Benefits and Limitations	10
2.8	1.8 Container Portability	10
2.9	1.9 Immutable and Ephemeral Infrastructure	10
3	PART 2 — Container History	12
4	PART 3 — Linux Technologies Behind Containers	14
4.1	3.1 Linux Namespaces	14
4.2	3.2 Linux cgroups (Control Groups)	15
4.3	3.3 Capabilities	15
4.4	3.4 Filesystems: OverlayFS and Copy-on-Write	15
5	PART 4 — Docker Fundamentals	16
5.1	4.1 Core Terms	16
5.2	4.2 Docker Architecture	16
5.3	4.3 What Actually Happens When You Run <code>docker run nginx</code>	17
6	PART 5 — Installing Docker (Ubuntu 24.04 LTS)	18
6.1	5.1 Installing Docker Engine	18
6.2	5.2 Post-Install: Run Docker Without <code>sudo</code>	18
6.3	5.3 Verifying the Installation	18
6.4	5.4 The Docker Daemon and Socket	19
7	PART 6 — The Complete Docker CLI	20
7.1	6.1 Images	20
7.2	6.2 Containers	20
7.3	6.3 System	22

7.4	6.4 Dangerous Commands, Explained in Full	22
8	PART 7 — docker run in Depth	23
8.1	7.1 The Chain docker run Sets Up	23
8.2	7.2 Core Flags, One at a Time	23
8.3	7.3 A Fully Worked, Realistic Example	24
9	PART 8 — Docker Images	25
9.1	8.1 Images and Layers	25
9.2	8.2 Image IDs, Tags, and Digests	25
9.3	8.3 Base Images and Minimal Images	25
9.4	8.4 Layer Caching (Preview of Part 29)	26
10	PART 9 — The Dockerfile	26
10.19.1	Every Core Instruction	26
10.29.2	CMD vs ENTRYPOINT	27
10.39.3	COPY vs ADD	27
10.49.4	ARG vs ENV	28
10.59.5	RUN vs CMD	28
10.69.6	A Complete, Realistic Dockerfile	28
11	PART 10 — Dockerfile Best Practices	30
11.110.1	The Core Checklist	30
11.210.2	.dockerignore	30
11.310.3	Bad vs Good Dockerfile, Side by Side	30
12	PART 11 — Multi-Stage Builds	32
12.111.1	The Problem They Solve	32
12.211.2	Go Example (Dramatic Size Difference)	32
12.311.3	Node.js Example	32
12.411.4	Python Example	33
12.511.5	Java Example	33
13	PART 12 — Docker Container Lifecycle	34
13.112.1	The States	34
13.212.2	Container PID 1	34
13.312.3	Exit Codes	34
13.412.4	Restart Policies	35
13.512.5	Signals and Graceful Shutdown	35
14	PART 13 — Docker Storage	36
14.113.1	The Writable Layer (Recap)	36
14.213.2	Volumes, Bind Mounts, tmpfs	36
14.313.3	Bind Mount vs Volume — When to Use Each	36
14.413.4	Database Containers and Persistent Storage	37
15	PART 14 — Docker Networking	37
15.114.1	Network Drivers	37
15.214.2	Why the Default Bridge Is Different from a Custom Bridge	38
15.314.3	Why Names, Not Hardcoded IPs	38
16	PART 15 — Port Mapping	39
16.115.1	-p HOST:CONTAINER	39
16.215.2	Variations	39
16.315.3	0.0.0.0 vs localhost, TCP vs UDP	39

16.415.4 EXPOSE vs Publish — A Critical Distinction	39
17PART 16 — Docker DNS	41
17.116.1 Docker's Embedded DNS Server	41
17.216.2 Service Discovery in Practice	41
17.316.3 Custom Networks and DNS Resolution Between Containers	41
18PART 17 — Docker Compose	41
18.117.1 What Compose Is and Why It Exists	41
18.217.2 The Top-Level Structure	42
18.317.3 A Complete Example	42
18.417.4 depends_on and Health Checks	43
18.517.5 Compose Commands	43
19PART 18 — Multi-Container Applications	44
19.118.1 Example 1: Nginx → Backend → PostgreSQL	44
19.218.2 Example 2: Nginx → Flask → Redis → PostgreSQL	45
20PART 19 — Environment Variables & Secrets	46
20.119.1 Ways to Set Environment Variables	46
20.219.2 Why Baking Secrets into an Image Is Dangerous	46
20.319.3 The Correct Alternatives	46
20.419.4 Practical Rule of Thumb	47
21PART 20 — Docker Registries	48
21.120.1 What a Registry Is	48
21.220.2 Authentication and the Push/Pull Flow	48
21.320.3 The Full Flow	48
22PART 21 — Docker Security	50
22.121.1 Root vs Non-Root	50
22.221.2 User Namespaces (Remapping)	50
22.321.3 Capabilities (Recap and Application)	50
22.421.4 Seccomp, AppArmor, SELinux	50
22.521.5 Read-Only Filesystem	50
22.621.6 Privileged Containers — Why --privileged Is Dangerous	51
22.721.7 Secrets (Recap Reference)	51
22.821.8 Image Scanning and Vulnerability Management	51
22.921.9 Supply-Chain Security (Preview of Part 40)	51
23PART 22 — Container Resource Management	52
23.122.1 Why Limits Matter	52
23.222.2 CPU Limits	52
23.322.3 Memory Limits	52
23.422.4 PIDs Limit	52
23.522.5 OOM Behavior	53
23.622.6 Monitoring Live Usage	53
24PART 23 — Health Checks	54
24.123.1 HEALTHCHECK in a Dockerfile	54
24.223.2 Liveness vs Readiness vs Startup (Conceptually)	54
24.323.3 Application Health Endpoints	54
24.423.4 Relationship to Kubernetes Probes	54
25PART 24 — Logging	56

25.124.1	stdout/stderr — The Docker Logging Model	56
25.224.2	Logging Drivers	56
25.324.3	Viewing Logs	56
25.424.4	Production Logging Architecture	57
26	PART 25 — Docker Monitoring	58
26.125.1	What docker stats Actually Measures	58
26.225.2	Beyond docker stats: Prometheus + Grafana	58
27	PART 26 — Docker Troubleshooting	59
27.126.1	Container Won't Start	59
27.226.2	Container Immediately Exits	59
27.326.3	Container Keeps Restarting (Crash Loop)	59
27.426.4	Image Won't Build	60
27.526.5	Image Pull Fails	60
27.626.6	Port Inaccessible	60
27.726.7	Container Cannot Reach Another Container	60
27.826.8	DNS Failure	61
27.926.9	Permission Denied	61
27.1026.10	Volume Doesn't Work / Data Isn't Persisting	61
27.1126.11	Disk Full (Host)	61
27.1226.12	Container Uses Too Much CPU	62
27.1326.13	Container Uses Too Much Memory	62
27.1426.14	Application Returns 500	62
27.1526.15	Nginx Returns 502	62
27.1626.16	Database Connection Fails	62
27.1726.17	Docker Daemon Fails	63
27.1826.18	Image Is Too Large	63
27.1926.19	Build Cache Not Working	63
27.2026.20	Container Cannot Access the Internet	63
28	PART 27 — Docker Debugging Commands	65
28.127.1	Docker-Level Tools	65
28.227.2	Connecting Back to Linux-Level Tools	65
29	PART 28 — Docker Performance	66
29.128.1	What Affects Performance	66
29.228.2	Optimization Strategies, by Factor	67
30	PART 29 — Docker Build Cache	68
30.129.1	How Caching Works	68
30.229.2	Why Changing One Line Can Invalidate Everything After It	68
30.329.3	General Structuring Principle	68
30.429.4	Verifying Cache Behavior	69
31	PART 30 — BuildKit	70
31.130.1	What BuildKit Is	70
31.230.2	Key Practical Features	70
31.330.3	Using BuildKit	70
32	PART 31 — Container Image Optimization: Choosing a Base Image	71
32.131.1	The Options, Compared	71
32.231.2	Security, Compatibility, Debugging, Size — The Real Trade-offs	71
32.331.3	When NOT to Use Alpine	72
32.431.4	A Practical Recommendation	72

33PART 32 — Docker with Python	72
33.132.1 The Chain	72
33.232.2 Virtual Environments vs Containers	73
33.332.3 requirements.txt and Reproducibility	73
33.432.4 A Production Flask/Gunicorn Dockerfile	73
33.532.5 Environment Variables and the Health Endpoint	73
34PART 33 — Docker with Nginx	75
34.133.1 The Pattern	75
34.233.2 Reverse Proxy Configuration	75
34.333.3 Serving Static Files	75
34.433.4 SSL/TLS Termination	76
34.533.5 Load Balancing	76
35PART 34 — Docker + CI/CD (Jenkins)	77
35.134.1 The Full Pipeline	77
35.234.2 A Complete Jenkinsfile	77
35.334.3 Why Tests and Scans Happen <i>Before</i> Push	78
36PART 35 — Docker + AWS	80
36.135.1 The Building Blocks	80
36.235.2 The Full Flow	80
36.335.3 EC2 + Docker (Self-Managed)	81
36.435.4 ECS/Fargate (Managed)	81
36.535.5 Docker and AWS EKS (Brief Preview of Part 37)	82
37PART 36 — Docker + Terraform	83
37.136.1 Different Responsibilities, Working Together	83
37.236.2 What Terraform Provisions for a Container Workload	83
37.336.3 A Minimal Terraform Example	83
37.436.4 The Boundary in Practice	84
38PART 37 — Docker + Kubernetes	84
38.137.1 The Relationship, Explicitly	84
38.237.2 Why Kubernetes Needs Container Images	85
38.337.3 OCI, containerd, CRI, runc — How Kubernetes Actually Runs a Container	85
38.437.4 Docker vs Kubernetes — Clearing Up the Confusion	85
38.537.5 The Historical “Docker vs Kubernetes” Relationship	86
39PART 38 — Docker Swarm (Brief)	87
39.138.1 What It Is	87
39.238.2 Architecture	87
39.338.3 Why Kubernetes Became More Widely Used	87
40PART 39 — OCI (Open Container Initiative)	88
40.139.1 Why It Exists	88
40.239.2 Practical Implications	88
40.339.3 Why Containers Are Not Limited to Docker	88
41PART 40 — Container Security & Supply Chain	89
41.140.1 The Supply-Chain Question	89
41.240.2 Base Image Security and Dependency Vulnerabilities	89
41.340.3 SBOM (Software Bill of Materials)	89
41.440.4 Image Signing and Provenance (Cosign)	89
41.540.5 SLSA (Supply-chain Levels for Software Artifacts)	89

41.640.6 Secrets, Least Privilege, Rootless Containers (Recap and Extension)	90
41.740.7 A Practical DevSecOps Checklist	90
42PART 41 — Production Best Practices Checklist	90
42.141.1 Image	90
42.241.2 Container Configuration	91
42.341.3 Secrets	91
42.441.4 Networking	91
42.541.5 Storage	91
42.641.6 Logging & Monitoring	91
42.741.7 CI/CD	92
42.841.8 Registry	92
42.941.9 Graceful Operation	92
43PART 42 — Hands-On Labs (50+)	93
43.1Foundations (Labs 1-10)	93
43.2Application Building (Labs 11-15)	93
43.3Compose & Multi-Container Apps (Labs 16-20)	93
43.4Resources, Logging, Security (Labs 21-30)	94
43.5Troubleshooting (Labs 31-35)	94
43.6CI/CD & Infrastructure (Labs 36-40)	94
43.7Production Patterns (Labs 41-50+)	94
44PART 43 — Complete Docker Cheat Sheet	95
44.1Installation	95
44.2Images	95
44.3Containers	96
44.4Dockerfile Instructions	96
44.5Volumes	96
44.6Networks	97
44.7Compose	97
44.8Registries	97
44.9Security	97
44.10Monitoring	97
44.11Troubleshooting	98
44.12System Cleanup	98
45PART 44 — Docker & Container Interview Preparation	98
45.1Docker Fundamentals	98
45.2Containers/Internals	98
45.3Images	99
45.4Dockerfile	99
45.5Networking	99
45.6Storage	100
45.7Compose	100
45.8Security	100
45.9Performance	100
45.10Troubleshooting (Scenario-Based)	100
45.11CI/CD	101
45.12AWS	101
45.13Kubernetes	101
45.14Container Internals	101
46PART 45 — Real DevOps Project: Full Production-Style Containerized Stack	102
46.1Project Goal	102

46.2Architecture	102
46.3Application	102
46.4Step-by-Step	102
47 Complete Docker Roadmap	104
48 30-Day Docker Learning Plan	105
48.1Daily Practice Schedule (~1.5-2 hrs/day)	106
49 Docker Troubleshooting Checklist (Print/Keep Handy)	106
50 Quick-Reference: Course Connections	106

1 How to Use This Course

This is a complete, from-zero-to-production reference on Docker and containers, built specifically for **DevOps/SRE work**. It is organized into 45 parts:

- **Parts 1-3:** What containers actually are, their history, and the Linux kernel technologies (namespaces, cgroups, OverlayFS) they're built on.
- **Parts 4-11:** Docker architecture, installation, the full CLI, `docker run`, images, and Dockerfiles (including multi-stage builds).
- **Parts 12-19:** Container lifecycle, storage, networking, Docker Compose, and secrets management.
- **Parts 20-25:** Registries, security, resource limits, health checks, logging, and monitoring.
- **Parts 26-31:** Troubleshooting, debugging, performance, build cache, and BuildKit.
- **Parts 32-37:** Real application patterns (Python, Nginx), CI/CD, AWS, Terraform, and Kubernetes.
- **Parts 38-41:** Docker Swarm, OCI, supply-chain security, and a full production checklist.
- **Parts 42-45:** 50+ hands-on labs, a command cheat sheet, interview prep, and a full capstone project.
- **Back matter:** roadmap, 30-day plan, and a printable troubleshooting checklist.

Notation used throughout:

- Commands are shown in monospace.
- **[DANGEROUS]** marks any command that can delete images/containers/volumes/data, modify networking, or expose the host — read the warning before running it.
- “DevOps use case” call-outs connect the raw Docker concept to what you'll actually do at work (CI/CD, AWS, Kubernetes).
- Every major concept ends with **common mistakes** and, where relevant, **troubleshooting**.

This course assumes you've worked through core Linux fundamentals (processes, namespaces, cgroups, networking) — where relevant it references that material directly, since Docker is, at its core, a well-packaged set of Linux kernel features.

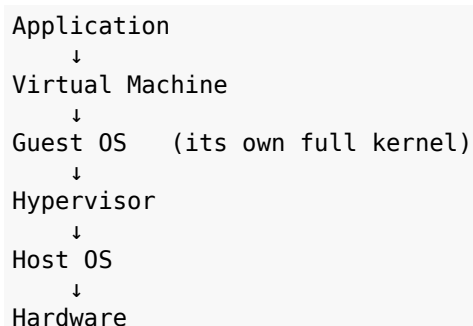
2 PART 1 — Container Fundamentals

2.1 1.1 What Is a Process?

A **process** is a running instance of a program — its own memory space, execution state, and (on Linux) a PID. Every container, ultimately, is just **one or more ordinary Linux processes**, given a restricted view of the system. There is no separate “container process type” in the kernel — this single fact demystifies almost everything else in this course.

2.2 1.2 What Is Virtualization? What Is a VM?

Virtualization creates an abstraction of hardware so multiple isolated operating systems can share one physical machine. A **virtual machine (VM)** is a full software emulation of a computer: its own kernel, its own OS, running on top of a **hypervisor** (e.g., VMware, KVM, or AWS’s Nitro hypervisor), which itself runs on the physical host.



2.3 1.3 What Is a Container?

A **container** is a Linux process (or group of processes) that has been given an isolated view of the system — its own filesystem, network stack, process tree, and hostname — using kernel features (namespaces, Part 3), and constrained in its resource usage (cgroups, Part 3). Crucially, **a container does not have its own kernel** — it shares the host’s kernel with every other process on the machine.



2.4 1.4 VM vs Container

	VM	Container
Isolation unit	Full hardware virtualization	Kernel namespaces + cgroups
Kernel	Own kernel per VM	Shared host kernel
Boot time	Seconds to minutes	Milliseconds to a couple seconds

	VM	Container
Size	GBs (full OS)	MBs to low GBs (just app + deps)
Density	Fewer per host	Many more per host
Isolation strength	Very strong (hardware-enforced)	Weaker (same kernel = larger shared attack surface)
Portability	Portable, but heavy	Highly portable, lightweight

2.5 1.5 Container vs Application, Container vs Image, Container vs Docker

- **Application:** your actual code/program (a Flask app, a compiled Go binary).
- **Image:** an inert, read-only **template** — a filesystem snapshot plus metadata (entrypoint, env vars, exposed ports) — built once, run many times. Think of it as a class.
- **Container:** a **running instance** of an image — a live process with its own writable layer on top. Think of it as an object instantiated from that class.
- **Docker:** one specific, extremely popular **tooling ecosystem** (CLI, daemon, image format, registry protocol) for building and running containers — not the only way containers can be created (Part 2, Part 39 — OCI/containerd/runc exist independently of Docker).

2.6 1.6 Why Containers Were Created

Before containers, the standard unit of deployment was either a full VM (heavy, slow to provision) or “just install it on the server” (fragile, “works on my machine” drift between dev/staging/prod). Containers solve this by packaging an application **with its exact dependencies** into one portable, immutable artifact that runs identically everywhere the container runtime is present — eliminating an entire category of environment-mismatch bugs.

2.7 1.7 Benefits and Limitations

Benefits: fast startup, high density (more workloads per host), consistent environments across dev/CI/prod, easy horizontal scaling, efficient resource usage, a clean unit for CI/CD and orchestration (Kubernetes).

Limitations: weaker isolation than VMs (shared kernel — a kernel-level exploit can potentially cross container boundaries), not ideal for workloads needing a genuinely different kernel/OS, stateful workloads need deliberate persistent-storage design (containers are meant to be disposable), and debugging a minimal container image can be harder (fewer tools installed by default).

2.8 1.8 Container Portability

Because an image bundles the application **and** all its runtime dependencies (libraries, interpreter, config defaults) as a single artifact, it behaves identically on a laptop, a CI runner, and a production Kubernetes node — as long as they’re running a compatible container runtime and a compatible kernel/architecture. This is the practical meaning of “works on my machine” being solved: the “machine” is now the container, not the host.

2.9 1.9 Immutable and Ephemeral Infrastructure

- **Immutable infrastructure:** once an image is built, it is never modified in place — any change means building a **new** image and replacing the running container(s), never patch-

ing a live one. This eliminates configuration drift.

- **Ephemeral infrastructure:** containers are treated as disposable — they can be killed and replaced at any time without ceremony. This is *why* state (databases, uploaded files) must live outside the container in a volume or external service (Part 13), never solely in the container's writable layer.

Common mistakes: treating a container like a lightweight VM you SSH into and hand-configure — this defeats immutability and portability; treat the Dockerfile as the single source of truth, and rebuild rather than patch.

3 PART 2 — Container History

Understanding this lineage explains *why* Docker looks the way it does, and why “Docker” and “containers” are not synonyms.

Era	Technology	What it added
1979	chroot	Change a process’s apparent root directory — the earliest filesystem isolation primitive, still not real containerization (no process/network isolation)
—	UNIX processes	The basic execution unit everything builds on
2002–2008	Linux namespaces	Real isolation — PID, network, mount, and other namespaces added incrementally to the kernel (Part 3)
2007	cgroups	Resource limiting/accounting, merged into the Linux kernel by Google engineers
2008	LXC (Linux Containers)	The first complete, user-friendly userspace tool combining namespaces + cgroups into “containers” as a coherent concept
2013	Docker	Built initially on LXC (later its own libcontainer), added a layered image format , a build system (Dockerfile), and a registry/distribution model (Docker Hub) — this combination is what made containers mainstream
2015	OCI (Open Container Initiative)	Docker and others standardized the image format and runtime spec, so containers aren’t locked to one vendor (Part 39)
2015–2016	containerd, runc	Docker split its internals into standalone, reusable components — runc (low-level, spec-compliant container execution) and containerd (higher-level container lifecycle management), both later donated to the CNCF

Era	Technology	What it added
2014–present	Kubernetes	Orchestrates many containers across many machines — scheduling, self-healing, scaling, service discovery — using these same OCI-compliant runtimes underneath

The key insight: Docker did not invent containers — it made them *usable*, by pairing existing kernel primitives with a great developer experience (a build file, a registry, a simple CLI). Kubernetes doesn't run "Docker" internally anymore in most modern clusters — it runs OCI-compliant runtimes (typically containerd) directly (Part 37).

4 PART 3 — Linux Technologies Behind Containers

This is the most important conceptual section in the course — everything else is built on these four mechanisms.

```
Linux
↓
Namespaces (isolate WHAT a process can see)
↓
cgroups (limit HOW MUCH a process can use)
↓
OverlayFS (layered, copy-on-write filesystem)
↓
Container Runtime (runc/containerd – assembles the above into "a container")
↓
Docker
```

4.1 3.1 Linux Namespaces

A **namespace** wraps a global kernel resource so a process inside it sees its own, isolated instance of that resource, while everything outside sees the normal, unmodified global one.

Namespace	Isolates	Container-relevant effect
PID	Process IDs	The container's main process is PID 1 <i>inside</i> the container, while being a normal PID on the host
NET	Network stack	Its own interfaces, IP address, routing table, port space — this is why two containers can each bind to port 8080 without conflict
MNT	Mount points	Its own filesystem view — the container "sees" only its image's filesystem, not the host's
UTS	Hostname/domain name	hostname inside a container returns the container ID, not the host's real name
IPC	Inter-process communication (shared memory, semaphores)	Prevents unrelated containers from sharing IPC resources
USER	UID/GID mapping	A process can appear as "root" inside its namespace while mapped to an unprivileged UID on the host — the basis of rootless containers (Part 21)
Cgroup	The cgroup hierarchy view itself	The container sees only its own slice of the cgroup tree, not the whole host's

```
ls -l /proc/<PID>/ns/           # inspect which namespaces a given process belongs to
docker inspect --format '{{.State.Pid}}' <container> # get a container's PID as seen
↳ from the HOST
```

This explains directly: why `ps aux` inside a container shows only that container's processes (PID namespace); why `docker exec` "enters" a container by joining its existing namespaces rather than starting something new; why two containers can have completely different, conflicting internal IP schemes without interfering with each other.

4.2 3.2 Linux cgroups (Control Groups)

cgroups enforce and account for **resource usage** — CPU, memory, PIDs, block I/O — for a group of processes. This is the exact mechanism behind Docker's `--cpus`, `--memory`, and `--pids-limit` flags (Part 22).

- **cgroup v1:** the original implementation, one hierarchy per resource controller (separate trees for CPU, memory, etc.) — still common on older systems.
- **cgroup v2:** the modern, unified hierarchy (one consistent tree for all controllers), default on Ubuntu 24.04 and current Docker — cleaner semantics, better delegation for containers.

```
cat /sys/fs/cgroup/memory.max # a container's memory limit, cgroup v2 view
cat /sys/fs/cgroup/cpu.max    # CPU quota/period
docker stats                  # Docker's friendly view of exactly this cgroup data
```

When a container exceeds its memory cgroup limit, the kernel's cgroup-scoped **OOM killer** terminates a process inside that cgroup specifically — this is the exact mechanism behind a container showing `OOMKilled: true`.

4.3 3.3 Capabilities

Traditionally a Linux process is either fully unprivileged or full **root**. **Capabilities** split root's power into ~40 fine-grained permissions (`CAP_NET_BIND_SERVICE`, `CAP_SYS_ADMIN`, `CAP_CHOWN`, etc.). Docker, by default, runs containers with a **reduced** capability set (not full root capabilities), and you can drop further capabilities explicitly.

- **Privileged containers** (`--privileged`) disable this protection entirely, granting **all** capabilities plus access to host devices — effectively removing the security boundary. Covered in depth, with a strong warning, in Part 21.
- **Least privilege:** drop every capability a container doesn't explicitly need (`--cap-drop=ALL --cap-add=<only what's needed>`).

4.4 3.4 Filesystems: OverlayFS and Copy-on-Write

Docker images are built from **layers** — each Dockerfile instruction typically produces one read-only layer. At runtime, the container gets a thin **writable layer** on top. **OverlayFS** (Docker's default storage driver, `overlay2`) merges all these layers into one coherent filesystem view using **copy-on-write**: reading a file comes from whichever layer has it; the *first write* to a file copies it up into the container's writable layer before modifying it, leaving the original image layers untouched.

```
Container writable layer (thin, ephemeral – deleted when the container is removed)
```

```
Image layer N (e.g., COPY . .)
Image layer 3 (e.g., RUN pip install ...)
```

```
Image layer 2 (e.g., RUN apt install ...)
Image layer 1 (e.g., FROM ubuntu:24.04)
```

This is exactly *why* multiple containers from the same image share disk space for their common layers, and why removing a container discards only its writable layer, leaving the underlying image untouched.

Common mistakes: believing a container is “a lightweight VM” — it’s a normal Linux process, isolated by namespaces, limited by cgroups, sharing the **exact same kernel** as the host; this is why you cannot run a container built for a different kernel architecture without a real VM/emulation layer underneath, and why kernel vulnerabilities are a genuine container-escape concern (motivating capabilities restrictions, seccomp, and stronger-isolation runtimes like gVisor/Kata for high-risk multi-tenant workloads).

5 PART 4 — Docker Fundamentals

5.1 4.1 Core Terms

- **Docker:** the overall tooling ecosystem for building, running, and sharing containers.
- **Docker Engine:** the core software that makes Docker work on a host — the daemon, API, and CLI together.
- **Docker daemon (dockerd):** the background service that does the actual work — building images, running containers, managing networks/volumes. A normal Linux process/systemd service (Part 8 of the Linux course).
- **Docker client (Docker CLI):** the docker command you type — a thin client that sends REST API requests to the daemon.
- **Docker API:** the REST API the daemon exposes over a Unix socket (`/var/run/docker.sock`) or, if configured, a TCP socket — everything the CLI does, it does by calling this API.
- **Docker Host:** the machine running the Docker daemon.
- **Docker Registry:** a server storing and distributing images (Docker Hub, Amazon ECR, private registries — Part 20).
- **Docker Image:** the read-only template (Part 1.5, Part 8).
- **Docker Container:** a running instance of an image (Part 1.5, Part 12).

5.2 4.2 Docker Architecture

```
Docker CLI (docker ...)
  ↓ (REST API call over a Unix socket)
Docker API
  ↓
Docker Daemon (dockerd) — high-level management: images, networks, volumes, build
  ↓
containerd — container lifecycle management (start/stop/pause), image
  ↪ pull/storage
  ↓
runc — actually creates the container: sets up namespaces,
  ↪ cgroups, and execs the process
  ↓
Linux Kernel
```

dockerd itself doesn’t directly create containers — it delegates to containerd, which delegates the low-level, spec-compliant creation step to runc (Part 2, Part 39). This layered architecture is exactly why Kubernetes can use containerd directly, without Docker in the middle at all

(Part 37).

5.3 4.3 What Actually Happens When You Run `docker run nginx`

1. Docker CLI sends a "create + start container" request to the Docker API.
2. dockerd checks: is the "nginx" image already present locally?
 - If not, it pulls it from the configured registry (Docker Hub by default), downloading each image layer.
3. dockerd asks containerd to create a container from that image.
4. containerd invokes runc, which:
 - a. Creates new namespaces (PID, NET, MNT, UTS, IPC) for the container's process.
 - b. Sets up a cgroup to enforce any resource limits.
 - c. Mounts the image's OverlayFS layers + a fresh writable layer.
 - d. Sets up networking – creates a veth pair, attaches one end to the Docker bridge network, assigns the container an internal IP.
 - e. execs the image's configured ENTRYPOINT/CMD (here, nginx's master process) as PID 1 inside the new namespaces.
5. dockerd streams the container's status back; containerd continues supervising the running process (restart policy, exit status).
6. The container is now a normal Linux process on the host – visible in ``ps aux`` on the host – just isolated and resource-limited via the mechanisms above.

This is Part 3's chain (namespaces → cgroups → OverlayFS → runtime) made concrete for one real command.

6 PART 5 — Installing Docker (Ubuntu 24.04 LTS)

6.1 5.1 Installing Docker Engine

```
# Remove any old/conflicting packages first
sudo apt remove -y docker docker-engine docker.io containerd runc 2>/dev/null

# Set up Docker's official APT repository
sudo apt update
sudo apt install -y ca-certificates curl gnupg
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o
  ↪ /etc/apt/keyrings/docker.gpg
sudo chmod a+r /etc/apt/keyrings/docker.gpg

echo \
  "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] \
  https://download.docker.com/linux/ubuntu $(. /etc/os-release && echo
  ↪ "$VERSION_CODENAME") stable" | \
  sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin
  ↪ docker-compose-plugin
```

This installs: **Docker Engine (docker-ce)**, the **Docker CLI**, **containerd.io** (the container runtime, Part 4.2), the **BuildKit-based buildx plugin** (Part 30), and the **Compose v2 plugin** (docker compose, Part 17 — note: modern Compose is a Docker CLI *plugin*, not the separate old docker-compose Python tool).

6.2 5.2 Post-Install: Run Docker Without sudo

```
sudo usermod -aG docker $USER      # add yourself to the docker group
newgrp docker                       # or log out/in for it to take effect
```

Security note: membership in the docker group is effectively **root-equivalent** on the host — a container can be run with a bind mount of / and full access to the host filesystem. Only grant this to trusted users/CI runners.

6.3 5.3 Verifying the Installation

```
docker version      # client AND server (daemon) version — confirms the daemon is
  ↪ actually reachable
docker info          # detailed daemon info: storage driver, cgroup version,
  ↪ number of containers/images, root dir
docker system info   # same as `docker info` (alias)
systemctl status docker # confirm the systemd-managed daemon service is active
sudo docker run hello-world # the canonical "it works" smoke test
```

docker info is one of the most useful first commands when troubleshooting a Docker host — it shows the **storage driver** (should be overlay2), **cgroup driver/version**, total images/containers, and the **Docker root directory** (/var/lib/docker by default, relevant when that filesystem fills up — Part 26).

6.4 5.4 The Docker Daemon and Socket

- `dockerd` runs as a `systemd` service (`docker.service`) — manage it exactly like any other `systemd` unit (Linux course Part 8): `systemctl start/stop/restart/status docker`.
- `docker.socket` — the Unix domain socket (`/var/run/docker.sock`) the daemon listens on by default; the CLI talks to this socket. Anyone with read/write access to this socket has full control of the daemon (again, root-equivalent) — this is *why* mounting the host's Docker socket into a container (`-v /var/run/docker.sock:/var/run/docker.sock`, “Docker-in-Docker via socket mounting”) is powerful **and** a significant privilege-escalation risk if that container is ever compromised.
- Daemon configuration lives in `/etc/docker/daemon.json` (e.g., to set the default logging driver, registry mirrors, or storage driver options) — changes require `sudo systemctl restart docker`.

Common mistakes: forgetting to add yourself to the `docker` group and reflexively prefixing every command with `sudo` (works, but is a sign you haven't finished setup — and running `docker` as `sudo` vs as a `docker-group` user can matter for file ownership on bind-mounted volumes); not realizing `docker group membership` = root-equivalent and granting it too liberally on a shared/multi-tenant host.

7 PART 6 — The Complete Docker CLI

7.1 6.1 Images

Command	Purpose	Key options	Example
<code>docker images /</code> <code>docker image ls</code> <code>docker pull</code>	List local images Download an image from a registry	<code>-a</code> all, <code>-q</code> quiet (IDs only) <code>IMAGE:TAG</code>	<code>docker images</code> <code>docker pull nginx:1.27</code>
<code>docker push</code>	Upload an image to a registry		<code>docker push myrepo/app:1.0</code>
<code>docker build</code>	Build an image from a Dockerfile	<code>-t</code> tag, <code>-f</code> Dockerfile path, <code>--no-cache</code>	<code>docker build -t myapp:1.0 .</code>
<code>docker image inspect</code> <code>docker image history</code>	Full JSON metadata for an image Show each layer and the command that created it		<code>docker image inspect nginx</code> <code>docker image history myapp</code>
<code>docker image rm /</code> <code>docker rmi</code>	Delete a local image	<code>-f</code> force	[DANGEROUS] <code>docker rmi myapp:old</code>
<code>docker image tag</code>	Add another tag to an existing image		<code>docker tag myapp:1.0 myapp:latest</code>
<code>docker save</code>	Export an image to a .tar file	<code>-o</code>	<code>docker save -o app.tar myapp:1.0</code>
<code>docker load</code>	Import an image from a .tar file	<code>-i</code>	<code>docker load -i app.tar</code>

```
docker images
# REPOSITORY TAG IMAGE ID CREATED SIZE
# nginx 1.27 a1b2c3d4e5f6 2 weeks ago 187MB
```

`docker image history myapp` — reading this output top-to-bottom shows exactly which Dockerfile instruction added how much size to the final image, invaluable when optimizing (Part 10, Part 28).

[DANGEROUS] `docker rmi $(docker images -q)` — deletes **every local image**. Any container currently using one will block the removal (safety net), but stopped containers won't. Always confirm with `docker images` first.

7.2 6.2 Containers

Command	Purpose	Key options	Example
<code>docker run</code>	Create and start a container (Part 7 in full)	many	<code>docker run -d -p 8080:80 nginx</code>
<code>docker create</code>	Create a container without starting it		<code>docker create --name app myapp</code>
<code>docker start</code>	Start an existing (stopped) container	<code>-a</code> attach	<code>docker start app</code>

Command	Purpose	Key options	Example
docker stop	Gracefully stop (SIGTERM, then SIGKILL after timeout)	-t seconds	docker stop app
docker restart	Stop then start		docker restart app
docker pause/unpause	Freeze/unfreeze all processes in a container (via cgroup freezer)		docker pause app
docker kill	Send SIGKILL (or a specified signal) immediately	-s	docker kill app
docker rm	Delete a stopped container	-f force (stops then removes), -v also remove anonymous volumes	[DANGEROUS] docker rm -f app
docker ps	List running containers	-a all incl. stopped, -q quiet	docker ps -a
docker inspect	Full JSON metadata (works for containers, images, networks, volumes)		docker inspect app
docker logs	View a container's stdout/stderr	-f follow, --tail	docker logs -f app
docker exec	Run a new command inside a running container	-it interactive	docker exec -it app bash
docker attach	Attach your terminal to the container's main process (PID 1)		docker attach app
docker cp	Copy files between host and container		docker cp app:/app/log.txt .
docker diff	Show filesystem changes vs the image (A=added, C=changed, D=deleted)		docker diff app
docker stats	Live resource usage (CPU/mem/net/IO)		docker stats
docker top	Show processes running inside a container (like ps, from the host)		docker top app
docker rename	Rename a container		docker rename app app-old
docker update	Change resource limits on a running container	--cpus --memory	docker update --memory=1g app

docker exec vs docker attach: exec starts a **brand-new** process inside the container's existing namespaces (most common for debugging — docker exec -it app bash gives

you a shell alongside the main process); attach connects to the **existing** PID 1 process’s own stdin/stdout/stderr — if that process isn’t expecting interactive input, attach can look “frozen,” and pressing Ctrl+C can accidentally kill the container’s main process (a common surprise for beginners). Prefer exec for debugging.

Common mistakes: using `docker rm -f` reflexively instead of `docker stop` — this skips the graceful SIGTERM shutdown window entirely; forgetting `-a` on `docker ps` and concluding a container “doesn’t exist” when it’s actually just stopped.

7.3 6.3 System

Command	Purpose	Example
<code>docker info</code>	Daemon-wide status/config	<code>docker info</code>
<code>docker version</code>	Client + server version	<code>docker version</code>
<code>docker system df</code>	Disk usage by images/containers/volumes/build cache	<code>docker system df -v</code>
<code>docker system prune</code>	Remove unused data (see danger note below)	[DANGEROUS]
<code>docker system events</code>	Live stream of daemon events (container start/stop/die, etc.)	<code>docker system events</code>

```
docker system df -v
# Shows exactly how much disk space images, containers, volumes, and the
# build cache are each consuming – the first command to run when a Docker
# host's disk is filling up (Part 26).
```

7.4 6.4 Dangerous Commands, Explained in Full

[DANGEROUS] `docker system prune` — removes **all stopped containers**, **all networks not used by at least one container**, **all dangling images** (untagged layers left over from rebuilds), and **all unused build cache**. Add `-a` to also remove **all images not used by any container** (not just dangling ones) — this can delete images you intended to keep for later. Add `--volumes` to **also delete unused volumes** — this **permanently destroys data** in any volume not currently attached to a running container. Always run `docker system df` first, and never run `--volumes` on a host with production data unless you are certain.

```
docker system prune          # safe-ish: stopped containers, dangling images,
↪ unused networks/build cache
docker system prune -a      # more aggressive: ALL unused images too
docker system prune -a --volumes # [DANGEROUS] – also deletes unused volumes and
↪ their DATA
```

[DANGEROUS] `docker rm -f <container>` — force-stops (SIGKILL, no graceful shutdown) and removes a container. Any data in its writable layer or unmounted anonymous volumes is gone.

[DANGEROUS] `docker rmi <image>` — deletes a local image; if it was the only copy and not pushed to a registry, it’s unrecoverable without rebuilding.

[DANGEROUS] `docker volume prune` — deletes every volume **not currently referenced by any container** (running or stopped). If you have a stopped-but-not-removed container still

referencing a volume, that volume is spared; anything else, including a volume you meant to keep for a future deploy, is deleted.

```
docker volume ls          # ALWAYS check first
docker volume prune      # [DANGEROUS] – deletes unreferenced volumes and their
↳ data, no undo
```

8 PART 7 — docker run in Depth

`docker run` is the single most-used Docker command — it combines `create` + `start` (+ optionally `attach`) into one step.

8.1 7.1 The Chain `docker run` Sets Up

```
Image
↓ (docker creates a container from it)
Container
↓ (runc execs the entrypoint)
Process (running as PID 1 inside the container's namespaces)
↓ (if -p was used)
Port (mapped from a host port to a container port)
↓ (attached to a Docker network)
Network
↓ (if -v/--mount was used)
Volume (persistent data outside the writable layer)
```

8.2 7.2 Core Flags, One at a Time

Basic execution

```
docker run nginx          # run in the FOREGROUND – your terminal attaches to the
↳ container's stdout
docker run -d nginx       # DETACHED – runs in the background, returns
↳ immediately, prints the container ID
docker run -it ubuntu bash # INTERACTIVE + TTY – allocates a pseudo-terminal and
↳ keeps stdin open;
                           # required for anything you want to type into
                           ↳ interactively (a shell, a REPL)
docker run --name webserver nginx # give the container a memorable, fixed name
↳ instead of a random one
```

`-it` is actually two flags: `-i` (`--interactive`, keep stdin open) and `-t` (`--tty`, allocate a pseudo-terminal). You need **both** for a usable interactive shell; `-i` alone with no `-t` leaves you without a proper prompt/line-editing.

Networking

```
docker run -p 8080:80 nginx # map HOST port 8080 -> CONTAINER port 80 (Part 15)
↳ covers this in full)
docker run -P nginx        # publish ALL exposed ports to random high host
↳ ports (rarely used in practice)
docker run --network mynet nginx # attach to a specific custom network instead of
↳ the default bridge (Part 14)
```

Environment and config

```
docker run -e DB_HOST=10.0.1.5 myapp          # set one environment variable
docker run --env-file .env myapp             # load many variables from a
↳ file (Part 19)
```

Storage

```
docker run -v mydata:/var/lib/postgresql/data postgres # named volume (Part 13)
docker run -v /host/path:/container/path myapp         # bind mount (Part 13)
docker run --mount type=volume,source=mydata,target=/data myapp # the newer, more
↳ explicit --mount syntax
```

Resource and identity control

```
docker run --restart unless-stopped myapp # restart policy (Part 12)
docker run --cpus=1.5 myapp              # limit to 1.5 CPU cores worth of time
docker run --memory=512m myapp           # hard memory limit
docker run --user 1000:1000 myapp         # run as a specific non-root UID:GID
↳ (Part 21)
```

8.3 7.3 A Fully Worked, Realistic Example

```
docker run -d \
  --name api-server \
  --restart unless-stopped \
  -p 8080:8080 \
  -e ENV=production \
  --env-file .env.prod \
  -v api-logs:/var/log/api \
  --network app-net \
  --cpus=1 \
  --memory=512m \
  --user 1000:1000 \
  myregistry/api:1.4.2
```

Reading this line by line is itself a mini-review of this whole part: run detached, name it, auto-restart unless manually stopped, publish port 8080, inject config via env vars, persist logs to a named volume, join a custom network for service discovery, cap resource usage, and drop root — this single command touches almost every concept in Parts 7, 12, 13, 14, 19, 21, and 22.

Common mistakes: forgetting `-d` and having a “detached” server appear to hang your terminal (it isn’t hung — it’s running in the foreground, exactly as asked); using `-p` without realizing it publishes to **all interfaces (0.0.0.0)** by default, potentially exposing a service to the whole network when only localhost access was intended (fix: `-p 127.0.0.1:8080:80`); mixing up host:container port order in `-p HOST:CONTAINER` (a very common transposition bug).

9 PART 8 — Docker Images

9.1 8.1 Images and Layers

An image is a stack of **read-only layers**, each corresponding to one Dockerfile instruction (roughly — some instructions like ENV/WORKDIR only change metadata and don't add a filesystem layer). Layers are **content-addressed** — identified by a SHA256 hash of their content — which enables powerful sharing: if two images share an identical base layer, Docker stores that layer's data **once** on disk, and reuses it for both.

```
Dockerfile
↓ docker build
Layers    (one per meaningful instruction, cached and content-addressed)
↓ merged via OverlayFS (Part 3.4)
Image     (the full read-only stack + metadata: entrypoint, env, exposed ports)
↓ docker run
Container (image + one thin writable layer on top)
```

9.2 8.2 Image IDs, Tags, and Digests

- **Image ID**: a SHA256 hash of the image's configuration (the actual unique identity of a specific image build).
- **Tag**: a human-friendly, **mutable** pointer to an image ID (e.g., nginx:1.27, myapp:latest) — a repository can have many tags, and a tag can be **re-pointed** to a different image later (this is exactly what happens on every new push of myapp:latest).
- **Digest**: a SHA256 hash of the image **manifest** — this is immutable and uniquely identifies one exact image build, regardless of what tags currently point to it.

```
docker pull nginx@sha256:abcdef1234...      # pull by DIGEST – guarantees you get the
↪ exact same bytes every time
docker inspect --format '{{.RepoDigests}}' nginx
```

latest is not special — it is just the default tag applied when you don't specify one, and by convention many projects push their newest build there. It is **not guaranteed to be the newest or most stable version** and should generally be avoided in production in favor of explicit version tags or, for maximum reproducibility, a pinned digest.

```
docker pull myapp:latest                    # ambiguous – could be a different image every time
↪ this is run
docker pull myapp:1.4.2                     # explicit semantic version – reproducible,
↪ human-readable
docker pull myapp@sha256:...                # digest pinning – byte-for-byte reproducible, the
↪ strongest guarantee
```

9.3 8.3 Base Images and Minimal Images

A **base image** is the FROM your Dockerfile starts from (Part 9). Choosing it well matters enormously for size, security, and compatibility — covered in full in Part 31 (Ubuntu vs Debian-slim vs Alpine vs distroless vs scratch).

Immutable images: once built and tagged with an immutable reference (a digest, or a version tag you never re-push), an image should never change — this is the image-layer half of “immutable infrastructure” (Part 1.9): fix a bug by building version N+1, not by mutating N.

9.4 8.4 Layer Caching (Preview of Part 29)

When building, Docker reuses a cached layer if the instruction **and everything before it** are unchanged — this is why instruction *order* in a Dockerfile has a major performance impact, covered fully in Part 29.

Common mistakes: treating `latest` as if it means “the newest stable release” — it’s just an unpinned mutable pointer; assuming two images with the same tag name on different registries/times are identical — always verify via digest when reproducibility genuinely matters (security audits, compliance, exact rollback).

10 PART 9 — The Dockerfile

A **Dockerfile** is a text file of instructions that `docker build` executes, top to bottom, each producing a new layer on top of the previous one.

10.1 9.1 Every Core Instruction

Instruction	Purpose	Example
FROM	Set the base image — must be the first real instruction	FROM python:3.12-slim
RUN	Execute a command at build time , its result committed as a new layer	RUN apt-get update && apt-get install -y curl
CMD	The default command run when the container starts (can be overridden at <code>docker run</code>)	CMD ["python3", "app.py"]
ENTRYPOINT	The command that always runs; args passed to <code>docker run</code> are appended to it	ENTRYPOINT ["python3", "app.py"]
COPY	Copy files from the build context into the image	COPY requirements.txt .
ADD	Like COPY, plus auto-extracts local tar archives and can fetch remote URLs	ADD app.tar.gz /app/
WORKDIR	Set the working directory for subsequent instructions (and the container’s default CWD)	WORKDIR /app
ENV	Set an environment variable, persists into the running container	ENV NODE_ENV=production
ARG	Define a build-time-only variable (Part 9.4)	ARG VERSION=1.0
EXPOSE	Document which port(s) the container listens on (metadata only — Part 15)	EXPOSE 8080
USER	Set the user (name or UID) subsequent instructions and the container run as	USER appuser

Instruction	Purpose	Example
VOLUME	Declare a mount point, signaling this path holds data that shouldn't live in the writable layer	VOLUME /var/lib/data
LABEL	Attach arbitrary metadata (key-value) to the image	LABEL main-tainer="devops@company.com"
HEALTHCHECK	Define how Docker checks if the container is actually healthy (Part 23)	HEALTHCHECK CMD curl -f http://localhost/health exit 1
SHELL	Change the default shell used for subsequent RUN/CMD/ENTRYPOINT shell-form instructions	SHELL ["/bin/bash", "-c"]

10.2 9.2 CMD vs ENTRYPOINT

Both define what runs when the container starts — the distinction is about **overridability**.

- **CMD**: the default command, fully **overridden** if you pass a command to `docker run` (`docker run myimage echo hi` replaces the CMD entirely with `echo hi`).
- **ENTRYPOINT**: the fixed command that **always executes**; anything passed to `docker run` is appended as **arguments** to it, not a replacement.

```
# Pattern 1: CMD only – fully replaceable default
CMD ["python3", "app.py"]
# docker run myimage          -> runs: python3 app.py
# docker run myimage bash     -> runs: bash    (CMD completely replaced)

# Pattern 2: ENTRYPOINT only – always runs, args appended
ENTRYPOINT ["python3", "app.py"]
# docker run myimage          -> runs: python3 app.py
# docker run myimage --debug   -> runs: python3 app.py --debug (appended, not
↳ replaced)

# Pattern 3 (very common, recommended): ENTRYPOINT + CMD together
ENTRYPOINT ["python3", "app.py"]
CMD ["--port", "8080"]
# docker run myimage          -> runs: python3 app.py --port 8080
# docker run myimage --port 9090 -> runs: python3 app.py --port 9090 (CMD's
↳ default args overridden)
```

Pattern 3 gives you a fixed, always-correct entrypoint binary/interpreter, with **overridable default arguments** — the best of both, and the most common real-world pattern for application images.

10.3 9.3 COPY vs ADD

- **COPY**: copies files/directories from the build context as-is. **Prefer this by default** — its behavior is simple and predictable.
- **ADD**: does everything COPY does, **plus** automatically extracts local `.tar/.tar.gz` archives into the destination, and can fetch a **remote URL** directly into the image.

```
COPY app.py /app/app.py          # simple, explicit file copy – preferred
ADD release.tar.gz /app/         # auto-extracts the archive on copy – a
↳ legitimate ADD use case
```

```
ADD https://example.com/file.txt /app/      # fetches a URL directly – generally
↳ discouraged (no build-time                # caching/checksum control; prefer an
                                             ↳ explicit RUN curl + verify)
```

Rule of thumb: use COPY unless you specifically need ADD's auto-extraction behavior.

10.4 9.4 ARG vs ENV

- **ARG:** a variable available **only during the build** (usable in RUN, other Dockerfile logic) — not present in the final running container unless explicitly also set via ENV.
- **ENV:** a variable set **in the image**, present both during later build steps **and** in every container started from that image (visible to the application, docker exec sessions, etc.).

```
ARG APP_VERSION=1.0.0                # build-time only
RUN echo "Building version $APP_VERSION"  # usable during build

ENV NODE_ENV=production                # persists into the running container
```

You can combine them to turn a build-time value into a runtime-visible one:

```
ARG APP_VERSION=1.0.0
ENV APP_VERSION=$APP_VERSION          # now baked into the image AND visible at runtime
```

```
docker build --build-arg APP_VERSION=2.1.0 -t myapp .    # override an ARG at build time
```

Security note: ARG values (including anything passed via --build-arg) are **not** a secure way to pass secrets — they can be inspected via docker history on the built image unless carefully handled. Never pass passwords/API keys as ARG/ENV (Part 19 covers this danger and the correct alternative in depth).

10.5 9.5 RUN vs CMD

- **RUN** executes **during the build**, and its result (filesystem changes) is committed as a permanent layer — used for installing packages, compiling code, creating directories.
- **CMD** does **not** execute during the build at all — it just records the default command to run **when a container starts** from the finished image.

```
RUN apt-get update && apt-get install -y curl    # happens ONCE, at build time, baked
↳ into the image
CMD ["python3", "app.py"]                        # happens EVERY TIME a container
↳ starts from this image
```

10.6 9.6 A Complete, Realistic Dockerfile

```
FROM python:3.12-slim
LABEL maintainer="devops@company.com"
WORKDIR /app

ARG APP_VERSION=1.0.0
ENV APP_VERSION=$APP_VERSION \
    PYTHONUNBUFFERED=1

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
```

```
COPY . .
```

```
RUN useradd -m appuser
```

```
USER appuser
```

```
EXPOSE 8080
```

```
HEALTHCHECK --interval=30s --timeout=3s CMD curl -f http://localhost:8080/health ||
```

```
↪ exit 1
```

```
ENTRYPOINT ["python3", "app.py"]
```

```
CMD ["--port", "8080"]
```

Common mistakes: using shell-form CMD `command arg` instead of exec-form CMD `["command", "arg"]` — shell form runs through `/bin/sh -c`, meaning your process is **not** PID 1 (the shell is), which breaks proper signal handling (Part 12.4) — always prefer exec (JSON array) form for CMD/ENTRYPOINT; putting USER `appuser` **before** an install step that needs root, causing a confusing permission-denied build failure.

11 PART 10 — Dockerfile Best Practices

11.1 10.1 The Core Checklist

- **Small images:** fewer layers of unnecessary tooling, smaller attack surface, faster pulls/deloys.
- **Layer/build-cache optimization:** order instructions from least-to-most frequently changing (Part 29).
- **.dockerignore:** exclude files from the build context you don't want copied in (and that would bust the cache unnecessarily).
- **Non-root user:** never run application containers as root in production (Part 21).
- **Multi-stage builds:** compile/build in one stage, ship only the runtime artifact (Part 11).
- **Minimal base images:** slim/Alpine/distroless where appropriate (Part 31).
- **Pin versions:** FROM python:3.12.4-slim, not FROM python:latest — reproducibility.
- **Avoid baking in secrets:** never ENV/ARG/COPY a credential into a layer (Part 19).
- **Reproducible builds:** pinned base image + pinned dependency versions (requirements.txt with exact versions, lockfiles) = the same build produces the same result every time.
- **Health checks:** let Docker/orchestrators know when a container is actually ready to serve traffic (Part 23).

11.2 10.2 .dockerignore

Exactly like .gitignore, but for the Docker build context:

```
.git
__pycache__/
*.pyc
.env
node_modules
*.md
tests/
.venv
```

Without this, COPY . . sends everything in the directory — including your entire .git history, local secrets, or node_modules — into the build context (slow) and potentially into the image itself (a real security/size problem).

11.3 10.3 Bad vs Good Dockerfile, Side by Side

Bad:

```
FROM ubuntu:latest
COPY . .
RUN apt-get update
RUN apt-get install -y python3 python3-pip
RUN pip3 install -r requirements.txt
ENV DB_PASSWORD=supersecret123
CMD python3 app.py
```

Problems: unpinned base (latest), copies everything before installing deps (busts cache on every code change), separate RUN lines that could be combined (more layers than needed), a secret baked into an ENV layer, CMD in shell form (breaks signal handling, Part 12.4), runs as root.

Good:

```
FROM python:3.12.4-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

RUN useradd -m appuser
USER appuser

EXPOSE 8080
CMD ["python3", "app.py"]
```

Fixes: pinned base image, dependency install happens **before** copying the rest of the source (so code-only changes don't bust the pip-install cache layer — Part 29), no secrets in the image, exec-form CMD, non-root user.

Common mistakes: one giant RUN line doing five unrelated things (hard to read, hard to cache selectively) versus five separate RUN lines for closely related steps (more layers than necessary) — the right balance is to **group logically related, always-together steps** into one RUN (e.g., apt-get update && apt-get install && apt-get clean in one layer, since splitting update and install across layers can cause stale-cache install failures), while keeping genuinely independent, differently-changing steps in separate RUN instructions.

12 PART 11 — Multi-Stage Builds

12.1 11.1 The Problem They Solve

Compiling/building an application often requires tools (compilers, build-time dependencies, source code) that are **completely unnecessary** at runtime — and bloat the final image if left in. **Multi-stage builds** let you use one (or more) temporary “build” stage with all that tooling, then copy **only the finished artifact** into a clean, minimal final stage.

```
Build Stage      (has compiler, dev dependencies, source code – large)
  ↓ compile
Compile
  ↓ produces
Artifacts        (a binary, compiled bytecode, static assets)
  ↓ COPY --from=build
Runtime Stage    (minimal base + just the artifact – small, shipped)
```

12.2 11.2 Go Example (Dramatic Size Difference)

```
# ---- Build stage ----
FROM golang:1.22 AS build
WORKDIR /src
COPY go.mod go.sum ./
RUN go mod download
COPY . .
RUN CGO_ENABLED=0 go build -o /app/server .

# ---- Runtime stage ----
FROM scratch
COPY --from=build /app/server /server
ENTRYPOINT ["/server"]
```

The golang:1.22 build image is well over 800MB; the final scratch-based image can be just the compiled binary — often under 20MB. scratch is literally an empty base (Part 31) — viable here because a statically-linked Go binary (CGO_ENABLED=0) needs no OS libraries at all.

12.3 11.3 Node.js Example

```
# ---- Build stage ----
FROM node:20 AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

# ---- Runtime stage ----
FROM node:20-slim
WORKDIR /app
COPY --from=build /app/dist ./dist
COPY --from=build /app/node_modules ./node_modules
COPY package*.json ./
USER node
CMD ["node", "dist/index.js"]
```

The full Node toolchain, dev dependencies, and raw source stay in the discarded build stage; only compiled output and production dependencies reach the final image.

12.4 11.4 Python Example

```
# ---- Build stage ----
FROM python:3.12 AS build
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir --target=/install -r requirements.txt

# ---- Runtime stage ----
FROM python:3.12-slim
WORKDIR /app
COPY --from=build /install /usr/local/lib/python3.12/site-packages
COPY . .
CMD ["python3", "app.py"]
```

Python doesn't compile to a binary the way Go does, but this pattern still helps when dependencies need build tools (e.g., packages with C extensions) that aren't needed once installed — the python:3.12 (non-slim) build stage has full build tooling; the runtime stage stays slim.

12.5 11.5 Java Example

```
# ---- Build stage ----
FROM maven:3.9-eclipse-temurin-21 AS build
WORKDIR /app
COPY pom.xml .
RUN mvn dependency:go-offline
COPY src ./src
RUN mvn package -DskipTests

# ---- Runtime stage ----
FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY --from=build /app/target/*.jar app.jar
ENTRYPOINT ["java", "-jar", "app.jar"]
```

The full Maven + JDK toolchain (large) never reaches the final image — only a JRE (smaller than a full JDK) plus the built .jar.

Why multi-stage builds matter: smaller final images (faster pulls, less attack surface, lower storage/registry cost), no leaked build-time secrets or source snippets left in intermediate layers of the *shipped* image, and a single Dockerfile stays the complete, reproducible build recipe rather than requiring a separate external build script.

Common mistakes: forgetting AS build naming and then being unable to reference the stage in COPY --from=; copying more than needed from the build stage (e.g., the whole /app including source and caches) instead of just the compiled artifact, silently reintroducing bloat.

13 PART 12 — Docker Container Lifecycle

13.1 12.1 The States

Created → Running → Paused → Stopped → Removed

- **Created:** `docker create` has allocated the container (filesystem, config) but not started its process yet.
- **Running:** the entrypoint process is executing as PID 1 inside the container's namespaces.
- **Paused:** all processes in the container are frozen (via the cgroup **freezer**) — no CPU time given, but memory state is preserved; `unpause` resumes exactly where it left off.
- **Stopped (Exited):** the main process has terminated (normally or via signal); the container's filesystem (writable layer) still exists on disk until removed.
- **Removed:** `docker rm` deletes the writable layer and the container's metadata — gone, unless data was in a separate volume.

```
docker ps -a          # see containers in ANY state, with their current status and exit
↪ code
```

13.2 12.2 Container PID 1

The container's main process runs as **PID 1** inside its own PID namespace (Part 3.1). This matters because PID 1 has special kernel responsibilities on any Linux system:

- It is expected to **reap zombie processes** (any orphaned child processes that finish) — if PID 1 doesn't do this, zombies accumulate inside the container.
- The kernel applies **default signal handling differently to PID 1** — for most signals, the default action (e.g., terminate) is only applied if PID 1 has explicitly installed a handler for that signal, or in some cases not applied by default at all. In practice: **a shell script or shell running as PID 1 often does not forward SIGTERM to child processes properly**, meaning `docker stop` (which sends SIGTERM) can fail to gracefully stop your actual application, forcing Docker to wait out the full timeout and then SIGKILL.

The exec fix, extremely common in entrypoint scripts:

```
#!/bin/bash
# BAD: this script stays as PID 1; the real app is a CHILD process,
# and won't reliably receive/forward SIGTERM.
python3 app.py

# GOOD: exec REPLACES the shell process with the app —
# the app itself becomes PID 1 and receives signals directly.
exec python3 app.py
```

This single `exec` keyword is one of the most impactful, most-overlooked details in production container images.

Minimal init systems: for cases where you can't avoid a shell/wrapper as PID 1 (e.g., you need to run setup logic before the real process), use a tiny init like `tini` (`docker run --init ...` or `ENTRYPOINT ["tini", "--", "python3", "app.py"]`) — it correctly reaps zombies and forwards signals on behalf of your real process.

13.3 12.3 Exit Codes

Exit code	Meaning
0	Success — clean exit
1-127 (non-zero)	Application-specific error (check the app's own logs)
137	128 + 9 — the container was SIGKILLED (often an OOM kill — check <code>docker inspect</code> for <code>OOMKilled: true</code>)
143	128 + 15 — the container received SIGTERM and exited (a graceful stop, or a shutdown that happened right at the timeout boundary)

```
docker inspect <container> --format '{{.State.ExitCode}} {{.State.OOMKilled}}'
```

13.4 12.4 Restart Policies

```
docker run --restart no myapp           # default: never auto-restart
docker run --restart on-failure myapp    # restart only on non-zero exit
docker run --restart on-failure:5 myapp  # restart on failure, max 5 attempts
docker run --restart unless-stopped myapp # always restart, EXCEPT if
↳ manually stopped by a user
docker run --restart always myapp        # always restart, even after a
↳ daemon/host reboot,
                                         # regardless of how it was stopped
```

`unless-stopped` is the most common production default for long-running services — it self-heals from crashes but respects an intentional `docker stop`.

13.5 12.5 Signals and Graceful Shutdown

```
docker stop <container>      # sends SIGTERM, waits (default 10s), then sends SIGKILL
↳ if still running
docker stop -t 30 <container> # give it 30 seconds instead of the default 10
docker kill <container>       # sends SIGKILL immediately — no grace period, no cleanup
```

Your application **must** handle `SIGTERM` (close DB connections, finish in-flight requests, flush buffers) within the grace period, or it will be forcibly killed mid-operation — the same principle covered for Kubernetes Pods in the Linux course (Part 29.3), because Kubernetes uses this exact same Docker/container-runtime signal mechanism underneath.

Common mistakes: shell-form `CMD/ENTRYPOINT` (Part 9.6) silently breaking signal forwarding; setting a restart policy of `always` on a container that's crash-looping due to a real bug — this can mask the underlying problem while continuously consuming resources on failed startup attempts (pair restart policies with alerting on restart *counts*, not just uptime).

14 PART 13 — Docker Storage

14.1 13.1 The Writable Layer (Recap)

Every running container has a thin, **ephemeral** writable layer (Part 3.4, OverlayFS) on top of its image's read-only layers. Any file the app writes there is **lost when the container is removed** — this is why anything that needs to persist (databases, uploads, logs you care about) must live in a **volume** or **bind mount** instead.

14.2 13.2 Volumes, Bind Mounts, tmpfs

Type	Where the data lives	Managed by Docker?	Typical use
Named volume	/var/lib/docker/volumes/<name>/_data (Docker-managed location)	Yes	Databases, any persistent app data — the recommended default
Anonymous volume	Same as named, but with an auto-generated name	Yes	Created implicitly by a Dockerfile VOLUME instruction or -v /container/path with no source; easy to lose track of
Bind mount	An exact path you choose on the host	No — you manage the host path yourself	Mounting source code for local dev live-reload, sharing specific host config files
tmpfs mount	RAM only, never touches disk	Yes (ephemeral)	Sensitive temp data (secrets during processing) that should never be written to disk at all

```
docker volume create mydata
docker volume ls
docker volume inspect mydata
docker volume rm mydata           # [DANGEROUS] permanently deletes the
    ↪ volume's data

docker run -v mydata:/var/lib/postgresql/data postgres           # named volume
docker run -v /home/aryan/src:/app myapp                         # bind mount — live host
    ↪ path
docker run --tmpfs /app/tmp myapp                                # RAM-only, gone on
    ↪ container stop
```

14.3 13.3 Bind Mount vs Volume — When to Use Each

- **Use a bind mount** when: you need Docker to see files that live in a specific, known location on the host — most commonly **local development** (mount your source code directory so code changes are reflected instantly without rebuilding), or mounting a specific host config file (/etc/localtime, a TLS cert) into a container.

- **Use a (named) volume** when: you need Docker-managed, portable, backup-friendly persistent storage — the standard choice for **databases** and any production persistent data. Volumes are decoupled from any specific host path, making them portable across hosts (especially relevant with volume drivers for network/cloud storage) and easier to back up (docker run --rm -v mydata:/data -v \$(pwd):/backup busybox tar czf /backup/backup.tar.gz /data).

14.4 13.4 Database Containers and Persistent Storage

```
docker volume create pgdata
docker run -d \
  --name postgres-db \
  -e POSTGRES_PASSWORD_FILE=/run/secrets/pg_password \
  -v pgdata:/var/lib/postgresql/data \
  --network app-net \
  postgres:16
```

The database’s actual data files live in the pgdata named volume — you can docker rm postgres-db and recreate the container from the same image, remounting pgdata, and the data survives intact. **Without** the volume, removing the container would permanently destroy the database.

Common mistakes: running a stateful database **without** any volume at all “just to test something quickly,” then losing real data when the container is inevitably removed/replaced during a routine deploy; confusing an **anonymous** volume (easy to accidentally orphan/lose track of — docker volume ls shows a long random hash instead of a friendly name) with a **named** one — always use named volumes for anything you intend to keep long-term; bind-mounting a host directory into a container running as a different UID and hitting confusing permission errors (the container’s UID must actually have permission on that host path).

15 PART 14 — Docker Networking

15.1 14.1 Network Drivers

Driver	Behavior	Typical use
bridge (default)	A private internal network on the host; containers get their own IP, NAT’d out for external access	Default for standalone containers on a single host
host	Container shares the host’s network namespace directly — no isolation, no port mapping needed (ports bind straight to the host)	Maximum network performance, single-host tools where isolation isn’t needed
none	No networking at all	Fully isolated batch jobs with zero network need
overlay	A virtual network spanning multiple Docker hosts (used by Docker Swarm, Part 38)	Multi-host container communication without Swarm/Kubernetes-level orchestration

Driver	Behavior	Typical use
custom bridge	A user-defined bridge network (docker network create) — like the default bridge, but with automatic DNS-based service discovery between containers (the default bridge does NOT provide this)	The recommended way to connect multiple containers on one host — always prefer this over the default bridge

```

docker network ls
docker network create app-net
docker network inspect app-net
docker network connect app-net some-container      # attach an already-running
↳ container to a network
docker network disconnect app-net some-container
docker network rm app-net                          # fails if any container is still
↳ attached

```

15.2 14.2 Why the Default Bridge Is Different from a Custom Bridge

Containers on the **default** bridge network (docker0) can reach each other **only by IP address** — no automatic DNS resolution by name. Containers on a **user-defined (custom) bridge** network get Docker's embedded DNS server for free, resolving **container/service names** to their current IP automatically. This is why virtually every real multi-container setup creates its own custom network rather than relying on the default bridge.

```

Container A (on custom network "app-net")
  ↓ connects to "backend:5000" (name, not IP — resolved by Docker's embedded DNS)
Docker Network ("app-net")
  ↓
Container B (named "backend", also on "app-net")

```

15.3 14.3 Why Names, Not Hardcoded IPs

A container's internal IP is **not stable** — it can change every time the container is recreated (a new deploy, a restart after a crash, a scaling event). Hardcoding an IP in application config will silently break the next time that container is replaced. **Always** connect between containers using their **container name** or (in Compose/Kubernetes) their **service name** — Docker's embedded DNS (or Kubernetes' own DNS, Linux course Part 29) resolves the name to whatever the current IP happens to be.

```

# BAD — will break the next time the database container is recreated with a new IP
DB_HOST = "172.18.0.3"

# GOOD — resolves via Docker's embedded DNS every time, always current
DB_HOST = "database"

```

16 PART 15 — Port Mapping

16.1 15.1 -p HOST:CONTAINER

```
docker run -p 8080:80 nginx
```

Host Port (8080) → Container Port (80)

External traffic arriving at the **host's** port 8080 is forwarded (via a NAT/iptables rule Docker manages automatically) to port 80 **inside** the container, where Nginx is actually listening.

16.2 15.2 Variations

```
docker run -p 8080:80 nginx                # bind to ALL host interfaces (0.0.0.0) –  
↳ reachable from anywhere  
docker run -p 127.0.0.1:8080:80 nginx      # bind ONLY to localhost – not reachable  
↳ from the network  
docker run -p 8080:80/udp nginx           # map a UDP port instead of the  
↳ (default) TCP  
docker run -p 8080:80 -p 9090:9090 nginx   # map multiple ports  
docker run -P nginx                       # publish EVERY EXPOSEd port to a  
↳ random unused host port
```

16.3 15.3 0.0.0.0 vs localhost, TCP vs UDP

- Binding to 0.0.0.0 (the default if you don't specify an IP in -p) makes the port reachable from **any network interface** on the host — including the public internet, if the host has a public IP and no firewall blocking it. This is a common accidental-exposure mistake.
- Binding to 127.0.0.1 restricts access to processes on the **host itself only** — appropriate when a port should never be reached externally (e.g., an internal admin interface you'll only ever reach via SSH tunnel).
- Most web/API traffic is **TCP**; **UDP** matters for protocols like DNS or some streaming/metrics protocols — specify with /udp if needed.

16.4 15.4 EXPOSE vs Publish — A Critical Distinction

```
EXPOSE 80
```

EXPOSE in a Dockerfile is **documentation and metadata only** — it does **not** open, forward, or publish anything. It tells humans (and tools like `docker run -P`) which port the containerized application listens on. **The only thing that actually makes a port reachable from outside the container is -p/-publish at docker run time** (or the ports: section in Compose, Part 17).

```
# EXPOSE 80 is in the Dockerfile, but this container is STILL unreachable from the host:  
docker run myimage
```

```
# THIS actually publishes it:  
docker run -p 8080:80 myimage
```

`docker run -P` (capital P, “publish all”) is the one place EXPOSE has functional effect — it auto-maps every EXPOSEd port to a random host port. Otherwise, treat EXPOSE purely as documentation.

Common mistakes: assuming `EXPOSE 80` in a Dockerfile means the app is reachable — it is not, until `-p` is used at runtime; transposing `HOST:CONTAINER` order in `-p`; forgetting that omitting an IP in `-p` binds to `0.0.0.0`, exposing a service more broadly than intended on a multi-tenant or internet-facing host.

17 PART 16 — Docker DNS

17.1 16.1 Docker's Embedded DNS Server

Every custom (user-defined) network gets an embedded DNS resolver, automatically available to every container on that network, resolving: - **Container names** (`--name`) to their current IP. - In Compose, **service names** (Part 17) — resolved the same way, since Compose creates a custom network per project by default.

17.2 16.2 Service Discovery in Practice

```
frontend                (container name: "frontend")
  ↓ HTTP request to http://backend:5000
backend                  (container name: "backend", same custom network)
  ↓ DB connection to postgresql://database:5432
database                 (container name: "database", same custom network)
```

```
docker network create app-net
```

```
docker run -d --name database --network app-net -e POSTGRES_PASSWORD=secret postgres:16
docker run -d --name backend --network app-net -e DB_HOST=database myapi:1.0
docker run -d --name frontend --network app-net -p 8080:80 -e
  ↪ API_URL=http://backend:5000 myui:1.0
```

Each container reaches the next purely by name — no IP ever hardcoded, no manual /etc/hosts editing. If database is ever recreated with a new internal IP, backend's next DNS lookup for database simply returns the new address — nothing else has to change.

17.3 16.3 Custom Networks and DNS Resolution Between Containers

Containers on **different** networks cannot resolve or reach each other by name (or at all, by default) — this is an intentional isolation boundary and a common source of “container A can't reach container B” troubleshooting (Linux course-style chain reasoning applies directly: are they even on the same network? `docker network inspect <net>` shows exactly which containers are attached).

```
docker network inspect app-net --format '{{range .Containers}}{{.Name}} {{end}}'
```

Common mistakes: two containers started with **no** `--network` flag both land on the default bridge, which provides **no** name-based DNS — engineers hit this constantly early on (“why can't my containers reach each other by name?”) before learning to always create and use a custom network; attaching a container to the wrong network entirely, or forgetting a service was ever attached to a shared network in the first place.

18 PART 17 — Docker Compose

18.1 17.1 What Compose Is and Why It Exists

Manually running several `docker run` commands with matching networks, volumes, and env vars for a multi-container application is repetitive and error-prone. **Docker Compose** lets you describe your **entire multi-container application** — services, networks, volumes, config — in one YAML file, then bring it all up (or down) with a single command. Modern Compose (`docker compose`, a CLI plugin — Part 5.1) is what's used today; the older standalone `docker-compose` (with a hyphen) is legacy.

18.2 17.2 The Top-Level Structure

```
services:      # the containers that make up your application
networks:      # custom networks (Compose creates a default one automatically even if
↳ you don't declare any)
volumes:       # named volumes for persistent data
configs:       # non-sensitive configuration files shared with services (newer
↳ Compose feature)
secrets:       # sensitive data, mounted rather than baked into env vars (Part 19.3)
```

18.3 17.3 A Complete Example

```
services:
  web:
    build: .
    ports:
      - "8080:80"
    environment:
      - API_URL=http://backend:5000
    depends_on:
      backend:
        condition: service_healthy
    networks:
      - app-net

  backend:
    image: myregistry/backend:1.4.2
    env_file:
      - .env.backend
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:5000/health"]
      interval: 10s
      timeout: 3s
      retries: 3
    depends_on:
      - database
    networks:
      - app-net

  database:
    image: postgres:16
    environment:
      POSTGRES_PASSWORD_FILE: /run/secrets/db_password
    secrets:
      - db_password
    volumes:
      - pgdata:/var/lib/postgresql/data
    networks:
      - app-net

networks:
  app-net:

volumes:
  pgdata:

secrets:
```

```
db_password:
  file: ./secrets/db_password.txt
```

Every service defined here becomes both a container **and** a DNS name (web, backend, database) automatically reachable from every other service on the same Compose-managed network — Part 16’s service discovery, with zero manual network setup.

18.4 17.4 depends_on and Health Checks

Plain `depends_on`: [database] only waits for the container to **start**, not for the application inside it to actually be **ready** (a database container can be “running” for a second or two before Postgres is actually accepting connections). `depends_on`: { condition: service_healthy } (shown above for web → backend) instead waits for the target service’s HEALTHCHECK (Part 23) to report healthy — the correct way to sequence genuinely dependent services.

18.5 17.5 Compose Commands

Command	Purpose	Example
<code>docker compose up</code>	Create and start all services	<code>docker compose up -d</code> (detached)
<code>docker compose down</code>	Stop and remove containers/networks (add <code>-v</code> to also remove volumes — [DANGEROUS] , deletes data)	<code>docker compose down</code>
<code>docker compose ps</code>	List this project’s containers and their status	<code>docker compose ps</code>
<code>docker compose logs</code>	View logs across all (or one) service	<code>docker compose logs -f backend</code>
<code>docker compose exec</code>	Run a command inside a running service’s container	<code>docker compose exec backend bash</code>
<code>docker compose build</code>	Build/rebuild images for services with a <code>build</code> : key	<code>docker compose build --no-cache</code>
<code>docker compose pull</code>	Pull the latest images for services with an <code>image</code> : key	<code>docker compose pull</code>
<code>docker compose restart</code>	Restart one or all services	<code>docker compose restart backend</code>
<code>docker compose config</code>	Validate and print the fully resolved configuration (great for debugging variable substitution)	<code>docker compose config</code>

```
docker compose up -d --build      # rebuild any local images, then start everything,
  ↳ detached
docker compose down -v           # [DANGEROUS] stop everything AND delete named
  ↳ volumes — data loss
```

Common mistakes: `docker compose down -v` on a host with a real database you meant to keep — always double-check before adding `-v`; using plain `depends_on` (start-order only) for a database dependency and hitting intermittent “connection refused” errors on cold start because the app started before Postgres was actually ready — fix with a proper healthcheck-based `depends_on` condition or in-app connection retry logic.

19 PART 18 — Multi-Container Applications

19.1 18.1 Example 1: Nginx → Backend → PostgreSQL

Nginx (reverse proxy + static files, port 80 published to the host)
↓
Backend (application server, internal-only, no published port)
↓
PostgreSQL (database, internal-only, no published port, persistent volume)

```
services:
  nginx:
    image: nginx:1.27
    ports:
      - "80:80"
    volumes:
      - ./nginx.conf:/etc/nginx/conf.d/default.conf:ro
    depends_on:
      - backend
    networks: [app-net]

  backend:
    build: ./backend
    env_file: [.env.backend]
    depends_on:
      database:
        condition: service_healthy
    networks: [app-net]

  database:
    image: postgres:16
    environment:
      POSTGRES_DB: appdb
      POSTGRES_PASSWORD_FILE: /run/secrets/db_password
    secrets: [db_password]
    volumes: [pgdata:/var/lib/postgresql/data]
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 5s
      retries: 5
    networks: [app-net]

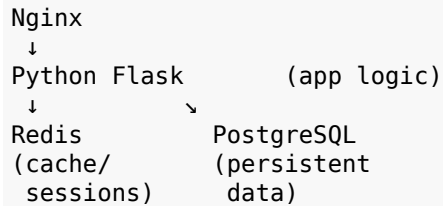
networks:
  app-net:

volumes:
  pgdata:

secrets:
  db_password:
    file: ./secrets/db_password.txt
```

Note: only nginx publishes a host port — backend and database are reachable **only from within app-net**, never directly from outside the host. This is the correct default security posture: minimize what's actually exposed (Part 21).

19.2 18.2 Example 2: Nginx → Flask → Redis → PostgreSQL



```
services:
  nginx:
    image: nginx:1.27
    ports: ["80:80"]
    volumes: [".:/nginx.conf:/etc/nginx/conf.d/default.conf:ro"]
    depends_on: [flask]
    networks: [app-net]

  flask:
    build: ./app
    environment:
      REDIS_HOST: redis
      DATABASE_URL: postgresql://appuser:@database:5432/appdb
    depends_on:
      redis:
        condition: service_started
      database:
        condition: service_healthy
    networks: [app-net]

  redis:
    image: redis:7-alpine
    networks: [app-net]

  database:
    image: postgres:16
    environment: {POSTGRES_DB: appdb, POSTGRES_USER: appuser}
    volumes: ["pgdata:/var/lib/postgresql/data"]
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U appuser"]
    networks: [app-net]

networks:
  app-net:
volumes:
  pgdata:
```

Flask connects to both redis and database purely by service name (Part 16.2); Redis here needs no persistent volume (typically used as an ephemeral cache — though redis *can* be configured with persistence if used as more than a cache).

Common mistakes: publishing a host port for every service “just in case” instead of only what genuinely needs external access — increases attack surface for no benefit; putting real secrets directly in environment: in the Compose file (Part 19) instead of using secrets:/env_file with a git-ignored file.

20 PART 19 — Environment Variables & Secrets

20.1 19.1 Ways to Set Environment Variables

```
docker run -e DB_HOST=10.0.1.5 myapp          # single variable, inline
docker run --env-file .env myapp              # load many variables from a file
```

```
# docker-compose.yml
services:
  app:
    environment:
      - DB_HOST=database
    env_file:
      - .env.app
```

.env file format:

```
DB_HOST=database
DB_PORT=5432
LOG_LEVEL=info
```

Compose also automatically reads a .env file in the project root for **variable substitution within the Compose file itself** (e.g., \${IMAGE_TAG}), which is a distinct mechanism from a service's env_file:.

20.2 19.2 Why Baking Secrets into an Image Is Dangerous

```
ENV PASSWORD=mysecret          # DANGEROUS — never do this
```

This is dangerous because: 1. **It's permanently baked into the image layer** — anyone who can pull or docker history the image can read it, even if a *later* layer tries to “unset” it (the earlier layer's data is still physically present). 2. **It gets pushed to the registry** — a secret meant for one environment is now sitting in your registry (Docker Hub, ECR) indefinitely, in every historical tag. 3. **It's visible via docker inspect** to anyone with access to a running container from that image.

```
docker history --no-trunc myimage | grep -i password  # this is exactly how easily a
↳ baked-in secret is found
```

20.3 19.3 The Correct Alternatives

Method	How it works	When to use
--env-file / Compose env_file	Injected at container start time , not baked into the image	Good for non-highly-sensitive config; still visible via docker inspect/docker exec env, so not ideal for top-secret credentials

Method	How it works	When to use
Docker secrets (Swarm/Compose secrets:)	Mounted as a file (e.g., /run/secrets/db_password) inside the container, not an env var — reduces accidental leakage via logs/env dumps/child-process inheritance	Better for real secrets in a Compose/Swarm context
AWS Secrets Manager / SSM Parameter Store	The application fetches the secret at runtime via an API call, authenticated via an IAM role (Linux course Part 27.2–27.3) — never stored on disk or in an image/env var at all	The strongest option for production AWS workloads
HashiCorp Vault	Similar dynamic-fetch model, cloud-agnostic	Multi-cloud/on-prem environments

```

services:
  database:
    image: postgres:16
    environment:
      POSTGRES_PASSWORD_FILE: /run/secrets/db_password  # tells Postgres to READ the
      ↪ secret from this file
    secrets:
      - db_password

secrets:
  db_password:
    file: ./secrets/db_password.txt  # this file itself must be .gitignore'd, never
    ↪ committed

```

20.4 19.4 Practical Rule of Thumb

- Non-sensitive config (feature flags, log level, service hostnames): plain environment:/env_file is fine.
- Sensitive credentials (DB passwords, API keys, TLS private keys): use secrets: (file-mounted) at minimum, or — for real production AWS workloads — fetch from **Secrets Manager/Parameter Store** at application startup using the instance/task's IAM role, so the secret never touches disk, an image layer, or a .env file at all.

Common mistakes: committing a .env file containing real secrets to Git (always add it to .gitignore, and double-check with `git status` before the first commit of a new project); assuming `--env-file` is “secure enough” because it's not baked into the image — it's still readable by anyone who can `docker inspect/docker exec` into that running container, which matters on any host or CI runner where you don't fully trust every other user/process.

21 PART 20 — Docker Registries

21.1 20.1 What a Registry Is

A **registry** stores and distributes images, organized into **repositories** (e.g., myregistry/backend), each holding multiple **tags** (versions) of that image.

Registry	Notes
Docker Hub	The default public registry; free tier has pull-rate limits for anonymous/unauthenticated use — a common source of “why did my CI build suddenly start failing” surprises
Amazon ECR (Elastic Container Registry)	AWS-native, IAM-authenticated, integrates directly with ECS/EKS (Part 35)
Private registries	Self-hosted (e.g., Harbor) or other cloud providers’ registries (GCR, ACR) — used when you need full control over access, retention, or air-gapped environments

21.2 20.2 Authentication and the Push/Pull Flow

```
docker login                                # Docker Hub (or pass -u/--password-stdin, or a
  ↳ specific registry hostname)
docker login <account>.dkr.ecr.<region>.amazonaws.com # ECR — typically via a
  ↳ short-lived token, see below

docker tag myapp:1.0 myregistry/myapp:1.0    # tag the LOCAL image with the registry's
  ↳ naming convention
docker push myregistry/myapp:1.0              # upload it
docker pull myregistry/myapp:1.0              # download it (elsewhere — another
  ↳ host, CI runner, prod server)
```

ECR authentication uses temporary, IAM-derived tokens rather than a static username/password:

```
aws ecr get-login-password --region us-east-1 | \
  docker login --username AWS --password-stdin
  ↳ <account-id>.dkr.ecr.us-east-1.amazonaws.com
```

This ties registry access directly to AWS IAM permissions (Linux course Part 27.2) — no separate registry credential to manage/rotate/leak.

21.3 20.3 The Full Flow

```
Developer
  ↓ git push
Docker Build (locally, or in CI — Part 34)
  ↓
Image (tagged with a version, e.g. myapp:1.4.2)
  ↓ docker push
Registry (Docker Hub / ECR / private)
  ↓ docker pull (from a server, ECS task, or Kubernetes node)
Deployment
```

Common mistakes: relying on anonymous Docker Hub pulls in CI and hitting rate limits during a busy build day (fix: authenticate, or use a registry mirror/cache); forgetting that docker tag doesn't rename/move an image — it adds an **additional** tag pointing at the same image ID, so the original local tag still exists too; pushing an image tagged latest and assuming every consumer automatically gets the update — they only do if they explicitly pull again, since latest is just a mutable pointer (Part 8.2).

22 PART 21 — Docker Security

22.1 21.1 Root vs Non-Root

By default, a container's main process runs as **root** *inside the container's user namespace* unless a Dockerfile sets `USER` or `docker run --user` overrides it. Because the container **shares the host kernel**, a process that's root inside the container **and** manages to escape the container's isolation (via a kernel vulnerability, a misconfiguration, or an overly permissive mount) could have real root-level impact on the host.

```
RUN useradd -m appuser
USER appuser
```

```
docker run --user 1000:1000 myapp    # or override at runtime
```

Always run application containers as a non-root user in production unless you have a specific, well-understood reason not to.

22.2 21.2 User Namespaces (Remapping)

Docker supports **user namespace remapping** at the daemon level (`/etc/docker/daemon.json`, `userns-remap` setting) — even a process that *is* root inside a container gets mapped to an **unprivileged** UID on the host, so a container-escape scenario doesn't grant real host root. This is an additional, daemon-wide layer of defense beyond just using `USER` in your Dockerfiles.

22.3 21.3 Capabilities (Recap and Application)

```
docker run --cap-drop=ALL --cap-add=NET_BIND_SERVICE myapp
```

Drop **all** default capabilities, then add back only the specific one(s) actually required (here, the ability to bind to a privileged port like 80) — the practical application of Part 3.3's least-privilege principle.

22.4 21.4 Seccomp, AppArmor, SELinux

- **Seccomp**: filters which **system calls** a container's processes are allowed to make at all — Docker applies a sane default seccomp profile automatically, blocking dozens of rarely-needed, higher-risk syscalls. Custom profiles can restrict further for high-security workloads.
- **AppArmor** / **SELinux**: mandatory access control (Linux course Part 21.3), applied to containers exactly as to any other Linux process — Docker on Ubuntu applies a default AppArmor profile to containers automatically; on RHEL-family hosts, SELinux contexts apply similarly.

```
docker run --security-opt seccomp=default.json myapp
docker run --security-opt apparmor=docker-default myapp
```

22.5 21.5 Read-Only Filesystem

```
docker run --read-only --tmpfs /tmp myapp
```

Makes the container's entire root filesystem **read-only** — the application can only write to explicitly provided tmpfs/volume mounts. This drastically limits what an attacker (or a bug)

can do even if code execution inside the container is achieved — a strong hardening measure for stateless services.

22.6 21.6 Privileged Containers — Why --privileged Is Dangerous

```
docker run --privileged myimage          # DANGEROUS
```

--privileged disables **almost every security mechanism covered above at once**: grants **all** Linux capabilities, disables the default seccomp filtering, disables AppArmor/SELinux confinement, and gives the container direct access to **host devices** (/dev). A process in a --privileged container can, in many configurations, directly access host hardware, load kernel modules, and in various documented scenarios **escape to the host entirely**. Legitimate uses are rare and narrow (certain low-level system tools, some CI/build scenarios needing Docker-in-Docker) — never use it for a normal application container, and never in a multi-tenant environment.

22.7 21.7 Secrets (Recap Reference)

Never bake secrets into images (Part 19.2) — use file-mounted Docker secrets or a real secrets manager (Part 19.3).

22.8 21.8 Image Scanning and Vulnerability Management

```
# Trivy – open-source vulnerability scanner, extremely common in CI pipelines
trivy image myapp:1.0
trivy image --severity HIGH,CRITICAL myapp:1.0    # focus on what actually matters for
↪ a release gate
```

Docker Scout (Docker’s own built-in scanning/analysis tooling, integrated into the Docker CLI/Hub) provides similar vulnerability and policy analysis directly within the Docker ecosystem.

```
docker scout cves myapp:1.0          # (illustrative – check current Docker Scout CLI
↪ syntax, it evolves)
```

The core practice: scan images **in CI, before they’re pushed to a registry or deployed** — catching a critical CVE in a base image or dependency before it ever reaches production, not after.

22.9 21.9 Supply-Chain Security (Preview of Part 40)

Where did this image’s base come from? What dependencies does it contain, and are any of them known-vulnerable or even maliciously tampered with? This broader question — covered fully in Part 40 — includes SBOMs, image signing (Cosign), and provenance attestation.

Common mistakes: using --privileged “just to make a networking/device issue go away” without understanding what security it disables; running every container as root by default because “it was easier to get working”; never scanning images for known CVEs until a security team flags it much later, well after the vulnerable image has been running in production for months.

23 PART 22 — Container Resource Management

23.1 22.1 Why Limits Matter

Without limits, a single misbehaving container (a memory leak, a runaway loop) can consume all of a host's CPU or RAM, starving **every other container** on that host — the containerized version of the Linux course's "noisy neighbor" problem (Linux course Part 12.3, Part 26.3), enforced here directly through **cgroups** (Part 3.2).

23.2 22.2 CPU Limits

```
docker run --cpus=1.5 myapp           # limit to 1.5 CPU cores' worth of
↳ processing time
docker run --cpu-shares=512 myapp      # RELATIVE weight vs other
↳ containers (default 1024) —
                                     # only matters when the host is
                                     ↳ under CPU contention;
                                     # doesn't cap usage when CPU is
                                     ↳ otherwise idle
docker run --cpuset-cpus="0,1" myapp   # pin the container to specific
↳ physical CPU cores
```

- **--cpus**: a hard **quota** (cgroup CPU quota/period) — the container genuinely cannot exceed this, even if the host is otherwise idle.
- **--cpu-shares**: a **relative priority weight**, not a hard cap — it only changes how CPU time is divided *when there's contention*; a container with double the shares of another gets roughly double the CPU time **only when both are competing** for it.

23.3 22.3 Memory Limits

```
docker run --memory=512m myapp        # hard memory limit — exceeding
↳ it triggers an OOM kill
docker run --memory=512m --memory-swap=1g myapp # allow up to 512m RAM + up
↳ to 512m swap (swap = memory-swap minus memory)
docker run --memory=512m --memory-reservation=256m myapp # a soft "target" the
↳ kernel tries to keep the
                                     # container near under host
                                     ↳ memory pressure,
                                     # without hard-enforcing
                                     ↳ it like --memory
                                     ↳ does
```

23.4 22.4 PIDs Limit

```
docker run --pids-limit=100 myapp
```

Caps the number of processes/threads a container can create — protects against **fork bombs** (accidental or malicious) that could otherwise exhaust the host's entire process table, affecting every other container and the host itself.

23.5 22.5 OOM Behavior

When a container exceeds its `--memory` limit, the kernel's **cgroup-scoped OOM killer** (Linux course Part 12.1, Part 26.3) terminates a process inside that container's cgroup specifically — other containers and the host are unaffected.

```
docker inspect <container> --format '{{.State.OOMKilled}}'    # confirm whether THIS
↳ was the cause of an exit
```

Exit code 137 (Part 12.3) is the most common visible symptom.

23.6 22.6 Monitoring Live Usage

```
docker stats                                # live view, all running containers
docker stats <container>                   # just one
docker stats --no-stream                   # a single snapshot instead of continuous
↳ refresh (useful in scripts)
```

CONTAINER	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O	BLOCK I/O	PIDS
backend	45.2%	380MiB / 512MiB	74.2%	1.2MB/3MB	0B/12MB	18

Production scenario: an application with a slow memory leak will show a **steadily climbing** MEM % over hours/days in `docker stats`/monitoring dashboards, eventually hitting its `--memory` limit and getting OOM-killed on a recurring cycle — the resource *limit* here is what turns an invisible leak into a visible, alertable, bounded (rather than host-destabilizing) symptom.

Common mistakes: setting no limits at all “because it’s simpler,” until one runaway container takes down a shared host; setting `--memory` far too low based on idle/light-load testing, causing OOM kills under real production traffic — always load-test with realistic traffic before finalizing limits; confusing `--cpu-shares` (relative, soft) with `--cpus` (absolute, hard) and being surprised a “limited” container still used 100% of a core when the host was otherwise idle.

24 PART 23 — Health Checks

24.1 23.1 HEALTHCHECK in a Dockerfile

```
HEALTHCHECK --interval=30s --timeout=3s --start-period=10s --retries=3 \
  CMD curl -f http://localhost:8080/health || exit 1
```

- **--interval**: how often to run the check.
- **--timeout**: how long to wait for the check command before considering it failed.
- **--start-period**: a grace period after container start during which failures **don't count** toward **--retries** — important for apps with a slow startup/warmup.
- **--retries**: how many consecutive failures before the container is marked unhealthy.

```
docker ps          # STATUS column shows "healthy"/"unhealthy"/"starting" once a
↳ HEALTHCHECK is defined
docker inspect <container> --format '{{.State.Health.Status}}'
```

24.2 23.2 Liveness vs Readiness vs Startup (Conceptually)

Docker's own HEALTHCHECK is a single, simple mechanism, but it's worth understanding the three distinct concepts orchestrators like Kubernetes split it into:

- **Liveness**: “is this process still working, or should it be restarted?” — a stuck/deadlocked process might still be *running* but no longer *functioning*.
- **Readiness**: “is this process currently able to accept traffic?” — a healthy process can still be temporarily *not ready* (e.g., still loading a large cache on startup, or intentionally draining connections before shutdown).
- **Startup**: “has this process finished its initial startup sequence?” — useful for slow-starting apps so liveness checks don't prematurely kill them before they've even had a chance to become ready.

Docker's single HEALTHCHECK most directly maps to **liveness**; **--start-period** approximates a **startup** grace window. It doesn't natively distinguish readiness — that's a Kubernetes-level addition (Part 37, Linux course Part 29).

24.3 23.3 Application Health Endpoints

A good `/health` endpoint should be **cheap and fast** (not itself doing expensive work) and should reflect *genuine* readiness — e.g., confirming a database connection pool is actually established, not just “the web server process is running”:

```
@app.route("/health")
def health():
    if not db.is_connected():
        return {"status": "unhealthy"}, 503
    return {"status": "healthy"}, 200
```

24.4 23.4 Relationship to Kubernetes Probes

Docker HEALTHCHECK	Kubernetes equivalent
Single check, "healthy/unhealthy"	→ livenessProbe (restart if failing)
--start-period	→ startupProbe (or initialDelaySeconds)

(no direct equivalent) → readinessProbe (remove from Service load-balancing pool without restarting the container)

Kubernetes doesn't actually read a Dockerfile's HEALTHCHECK directive at all — it defines its own probes (livenessProbe, readinessProbe, startupProbe) directly in the Pod spec, hitting the **same** application health endpoints you'd build for Docker's HEALTHCHECK. Building a good /health endpoint once serves both.

Common mistakes: a HEALTHCHECK/health endpoint that only checks “is the HTTP server up” without checking real downstream dependencies (database, cache) — masks genuine failures; too-short --start-period/initialDelaySeconds on a slow-starting app, causing repeated unnecessary restarts right as it's finishing startup (a restart loop caused by the health check itself, not a real problem).

25 PART 24 — Logging

25.1 24.1 stdout/stderr — The Docker Logging Model

Docker’s logging model is intentionally simple: whatever your container’s main process writes to **stdout** and **stderr** is captured by the configured **logging driver**. Applications should log there directly, **not** to files inside the container (which would be lost with the writable layer and invisible to `docker logs`) — this is the container-world equivalent of “twelve-factor app” logging philosophy.

25.2 24.2 Logging Drivers

Driver	Behavior
json-file (default)	Writes logs as JSON to a file on the host, under Docker’s data directory — simple, works out of the box, but can grow unbounded without log rotation config
local	A more efficient, Docker-optimized on-disk format with built-in rotation by default
syslog	Sends logs to the host’s (or a remote) syslog daemon (Linux course Part 9.1)
journald	Sends logs into systemd’s journal (Linux course Part 9.1), queryable via <code>journalctl CONTAINER_NAME=...</code>
fluentd	Ships logs to a Fluentd collector — common in centralized-logging production setups feeding into Elasticsearch/OpenSearch, S3, or similar

```
// /etc/docker/daemon.json
{
  "log-driver": "json-file",
  "log-opts": {
    "max-size": "10m",
    "max-file": "3"
  }
}
```

Setting `max-size`/`max-file` is essential in production with the default `json-file` driver — without it, a chatty container’s logs can grow without bound and fill the host’s disk (the exact “logs fill the disk” incident class from the Linux course, Part 11.4/30.21, now scoped to Docker’s log storage specifically).

25.3 24.3 Viewing Logs

```
docker logs <container>          # all captured logs so far
docker logs -f <container>       # follow, live (like tail -f)
docker logs --tail 100 <container> # last 100 lines only
docker logs --since 10m <container> # only the last 10 minutes
```

25.4 24.4 Production Logging Architecture

```
Container (stdout/stderr)
  ↓
Docker logging driver (json-file / journald / fluentd / ...)
  ↓
Log shipper / agent (Fluent Bit, CloudWatch Agent, Filebeat...)
  ↓
Centralized log store (CloudWatch Logs, Elasticsearch/OpenSearch, Loki, Datadog...)
  ↓
Dashboards / alerting (Kibana, Grafana, CloudWatch Insights...)
```

In production, you rarely docker logs your way through an incident on a live host at scale — logs are shipped centrally so they're searchable across every container/host/deployment, and survive even after the container that generated them is long gone.

Common mistakes: letting the default json-file driver grow unbounded with no max-size/max-file set, eventually filling the host's disk; writing application logs to a file inside the container instead of stdout/stderr, making them invisible to docker logs and any log-shipping pipeline built around Docker's standard model.

26 PART 25 — Docker Monitoring

26.1 25.1 What docker stats Actually Measures

```
docker stats
```

Metric	Source (cgroup)
CPU %	cgroup CPU accounting (cpu.stat)
Memory usage/limit	cgroup memory controller (memory.current, memory.max)
Network I/O	Per-container network namespace counters
Block I/O	cgroup io controller (disk read/write)
PIDs	cgroup pids.current

This is the same cgroup data from Part 3.2 and Part 22 — `docker stats` is simply Docker's friendly, real-time presentation of numbers the kernel is already tracking.

26.2 25.2 Beyond docker stats: Prometheus + Grafana

`docker stats` is great for a quick, live, single-host check — but it isn't historical, isn't queryable, and doesn't alert. Production monitoring typically looks like:

```
Docker (containers, cgroup metrics)
  ↓ (cAdvisor exports container-level metrics; node_exporter exports host-level
  ↪ metrics)
Prometheus (scrapes and stores time-series metrics)
  ↓
Grafana      (dashboards, visualization)
  ↓
Alertmanager (routes alerts derived from Prometheus rules to Slack/PagerDuty/email)
```

- **cAdvisor** (Container Advisor): exposes per-container resource metrics (the same cgroup data `docker stats` reads) in a format Prometheus can scrape.
- **node_exporter**: exposes host-level metrics (CPU, memory, disk, network at the OS level — Linux course Parts 12-14).
- **Prometheus**: a time-series database that periodically **scrapes** metrics endpoints and stores them, with a query language (PromQL) for building alerts/dashboards.
- **Grafana**: visualizes Prometheus (and many other) data sources as dashboards.

docker-compose.yml snippet — a minimal monitoring stack

```
services:
  cAdvisor:
    image: gcr.io/cadvisor/cadvisor:latest
    ports: ["8081:8080"]
    volumes:
      - /:/rootfs:ro
      - /var/run:/var/run:ro
      - /sys:/sys:ro
      - /var/lib/docker:/var/lib/docker:ro

  prometheus:
    image: prom/prometheus
    ports: ["9090:9090"]
    volumes: ["/etc/prometheus/prometheus.yml"]
```

```
grafana:
  image: grafana/grafana
  ports: ["3000:3000"]
```

Common mistakes: relying only on `docker stats` for production monitoring — fine for ad-hoc debugging, but offers no history, no alerting, and requires someone actively watching a terminal; forgetting that container metrics alone don't tell the whole story — always correlate with **host-level** metrics (Linux course Parts 12–13) since host-wide resource exhaustion affects every container simultaneously in ways per-container metrics alone can obscure.

27 PART 26 — Docker Troubleshooting

Each scenario follows: **Symptoms** → **Possible Causes** → **Commands** → **Interpretation** → **Root Cause** → **Fix** → **Prevention**.

27.1 26.1 Container Won't Start

- **Symptoms:** `docker run/docker start` returns an error immediately, or the container never reaches `Running`.
- **Causes:** invalid command/entrypoint, missing image layer, port already in use, invalid volume/mount path, malformed config.
- **Commands:** `docker run` (watch the immediate error output), `docker logs <container>`, `docker inspect <container>`.
- **Root cause (example):** `-p 8080:80` but port 8080 is already bound by another process on the host.
- **Fix:** free the port, or map to a different host port.
- **Prevention:** standardize port allocation across services; check `ss -tulnp` (Linux course Part 14.8) before assuming a port is free.

27.2 26.2 Container Immediately Exits

- **Symptoms:** `docker ps` shows nothing; `docker ps -a` shows `Exited (n)` moments after start.
- **Commands:** `docker logs <container>`, `docker inspect --format '{{.State.ExitCode}}' <container>`.
- **Root cause (example):** the entrypoint process completed and returned immediately (e.g., you ran a one-shot command like `ls`, not a long-running server) — containers exit as soon as their PID 1 process exits, by design.
- **Fix:** confirm the intended main process is actually long-running (a server loop, not a script that finishes).
- **Prevention:** for debugging, use `docker run -it <image> bash` to interactively confirm what the entrypoint actually does before running detached.

27.3 26.3 Container Keeps Restarting (Crash Loop)

- **Symptoms:** `docker ps` shows repeated `Restarting` status.
- **Commands:** `docker logs <container>` (the actual crash reason), `docker inspect --format '{{.State.OOMKilled}} {{.State.ExitCode}}' <container>`.
- **Root cause (example):** a missing required environment variable causes the app to exit on startup, and `--restart unless-stopped` keeps retrying forever.

- **Fix:** supply the missing configuration; check `docker compose config` if using Compose, to verify variable substitution actually resolved as expected.
- **Prevention:** startup config validation with a clear, loggable error message; alert on restart **count**, not just container “up” status.

27.4 26.4 Image Won’t Build

- **Symptoms:** `docker build` fails mid-way.
- **Commands:** read the exact failing instruction/line in the build output; `docker build --no-cache` to rule out a stale-cache issue; `docker build --progress=plain` for full, unbuffered output.
- **Root cause (example):** a `RUN apt-get install` step fails because an earlier cached `apt-get update` layer is now stale (package index out of date relative to what’s being installed).
- **Fix:** combine update and install into the same `RUN` layer (Part 10.3) so they’re never cached independently; or `--no-cache` to force a fresh build.
- **Prevention:** always chain `apt-get update && apt-get install` in one `RUN`; pin dependency versions to reduce “worked yesterday, fails today” build drift.

27.5 26.5 Image Pull Fails

- **Symptoms:** `docker pull/docker run` fails to retrieve the image.
- **Commands:** `docker pull <image>` directly (isolate from whatever wrapped it), check registry auth (`docker login`), check network/DNS from the host, check for rate limiting (Docker Hub anonymous pulls, Part 20.1).
- **Root cause (example):** an expired ECR auth token (they’re short-lived, typically 12 hours).
- **Fix:** re-authenticate (`aws ecr get-login-password | docker login ...`).
- **Prevention:** automate registry re-authentication as a standard step in CI/CD pipelines and long-running deploy scripts, not a one-time manual step.

27.6 26.6 Port Inaccessible

- **Symptoms:** can’t reach a service that should be published.
- **Commands:** `docker ps` (confirm the port mapping shown, e.g. `0.0.0.0:8080->80/tcp`), `ss -tulnp` on the host (Linux course Part 14.8), host firewall/Security Group check, `curl localhost:8080` from the host itself first (isolate host-level vs external-network issues).
- **Root cause (example):** `EXPOSE` was set in the Dockerfile but no `-p` was passed at run-time (Part 15.4) — a very common confusion.
- **Fix:** add the correct `-p HOST:CONTAINER` mapping.
- **Prevention:** treat `EXPOSE` as documentation only, always verify actual publishing with `docker ps`.

27.7 26.7 Container Cannot Reach Another Container

- **Symptoms:** connection refused/timeout between two containers.
- **Commands:** `docker network inspect <network>` (are both containers actually attached?), `docker exec <container> ping <other-container-name>`, confirm both are using **container/service names**, not stale IPs (Part 14.3).
- **Root cause (example):** the two containers were started without `--network`, landing separately on the default bridge with no name-based DNS (Part 16.3).
- **Fix:** create and use a custom network for both.

- **Prevention:** always define an explicit custom network in Compose/docker run, never rely on the default bridge for multi-container communication.

27.8 26.8 DNS Failure

- **Symptoms:** a container can't resolve a hostname (either another container's name, or an external domain).
- **Commands:** `docker exec <container> cat /etc/resolv.conf`, `docker exec <container> nslookup <name>`, confirm the container is on the network that provides that DNS entry.
- **Root cause (example):** the container is on the wrong (or default) network for internal name resolution, or the host's own DNS resolver is unreachable for external lookups.
- **Fix:** attach to the correct custom network; verify host DNS connectivity separately.
- **Prevention:** consistent network naming/attachment across all Compose services.

27.9 26.9 Permission Denied

- **Symptoms:** the app inside the container can't read/write a file.
- **Commands:** `docker exec <container> ls -l <path>`, `docker exec <container> id`, check the UID the container runs as (USER/--user) against the host-path ownership for bind mounts.
- **Root cause (example):** a bind-mounted host directory is owned by a UID that doesn't match the container's non-root USER.
- **Fix:** align UIDs (chown the host path to match, or set --user to match the host path's owning UID), or use a named volume instead where Docker manages ownership more transparently.
- **Prevention:** document/standardize the UID your images run as; be deliberate about bind-mount ownership in local dev setups.

27.10 26.10 Volume Doesn't Work / Data Isn't Persisting

- **Symptoms:** data disappears after a container restart/recreation.
- **Commands:** `docker inspect <container>` (check the Mounts section — is a volume actually attached at the path the app writes to?), `docker volume ls`.
- **Root cause (example):** the app writes to /data, but the volume was mounted at /app/data — a path mismatch.
- **Fix:** align the volume mount path exactly with where the application actually writes.
- **Prevention:** verify mount paths against actual application config, not assumption; test persistence explicitly (write data, recreate the container, confirm it's still there) before considering storage "done."

27.11 26.11 Disk Full (Host)

- **Symptoms:** builds/pulls/writes fail with "no space left on device."
- **Commands:** `docker system df -v` (what's actually consuming space — images, containers, volumes, or build cache), `df -h` on the host (Linux course Part 10.3, 11.1).
- **Root cause (example):** years of accumulated dangling images and unused build cache never cleaned up.
- **Fix:** `docker system prune` (carefully — Part 6.4's danger notes apply in full).
- **Prevention:** scheduled, routine `docker system prune` (non--a, non---volumes for automated jobs unless you're certain), and disk usage alerting on hosts running Docker.

27.12 26.12 Container Uses Too Much CPU

- **Symptoms:** `docker stats` shows sustained high CPU% for one container.
- **Commands:** `docker stats`, `docker exec <container> top` or `docker top <container>` (from the host, no exec needed), then dig into the app itself (profiling, `strace` from the host targeting the container's host-visible PID).
- **Root cause (example):** an inefficient loop/regex, or genuinely more traffic than the container's CPU limit accommodates.
- **Fix:** optimize the code, or raise `--cpus` (with capacity planning, not as a blind reflex).
- **Prevention:** load testing before production, CPU limits set based on realistic measured usage, autoscaling where the platform supports it.

27.13 26.13 Container Uses Too Much Memory

- **Symptoms:** climbing MEM % in `docker stats`, eventual OOM kill (exit code 137).
- **Commands:** `docker stats`, `docker inspect --format '{{.State.OOMKilled}}' <container>`.
- **Root cause (example):** a memory leak in a long-running process.
- **Fix:** restart short-term (buys time), profile and fix the leak long-term.
- **Prevention:** memory limits set deliberately (Part 22.3), alerting on memory trend (not just threshold), regular restarts as a stopgap only while the real fix is in progress (never a permanent substitute for fixing a leak).

27.14 26.14 Application Returns 500

- **Symptoms:** the app itself responds, but with a server error.
- **Commands:** `docker logs <container>` for the actual stack trace/exception.
- **Root cause (example):** an unhandled exception on a specific input, or a downstream dependency (database) being unreachable.
- **Fix:** patch the bug or restore the dependency.
- **Prevention:** better error handling, dependency health checks surfaced clearly in logs, structured logging for faster diagnosis.

27.15 26.15 Nginx Returns 502

- **Symptoms:** the Nginx **container** responds, but with 502.
- **Commands:** confirm the upstream container is actually running (`docker ps`), confirm both are on the same network, `docker exec nginx-container curl http://backend:5000` to isolate.
- **Root cause (example):** the backend container crashed or hasn't started yet when Nginx tried to proxy to it.
- **Fix:** fix/restart the backend; add a proper `depends_on` health-check condition (Part 17.4) so Nginx isn't started before its upstream is genuinely ready.
- **Prevention:** health-check-gated startup ordering, Nginx-level retry/upstream failover configuration.

27.16 26.16 Database Connection Fails

- **Symptoms:** the app can't connect to its database container.
- **Commands:** `docker exec app-container nc -zv database 5432` (or the relevant port) to isolate connectivity from application-level auth issues, `docker logs database-container` for the DB's own startup status.

- **Root cause (example):** the app started before Postgres finished its own internal initialization (first-run schema setup can take longer than a plain “container started” signal implies).
- **Fix:** use a proper health-check-based `depends_on` (Part 17.4), or in-app retry-with-backoff connection logic (more robust, works even outside Compose).
- **Prevention:** never assume “container running” means “service ready” — always health-check the actual service protocol/port.

27.17 26.17 Docker Daemon Fails

- **Symptoms:** any docker command errors out immediately (Cannot connect to the Docker daemon).
- **Commands:** `systemctl status docker` (Linux course Part 8.4), `journalctl -u docker -n 100`.
- **Root cause (example):** the daemon failed to start due to a malformed `/etc/docker/daemon.json`.
- **Fix:** correct the JSON syntax, `sudo systemctl restart docker`.
- **Prevention:** validate `daemon.json` changes (`docker daemon.json` is strict JSON — trailing commas etc. will break it) before restarting the service on a production host, ideally via config management/CI validation rather than manual live edits.

27.18 26.18 Image Is Too Large

- **Symptoms:** slow pulls/pushes, high registry storage cost.
- **Commands:** `docker image history <image>` (which layer added the most size), `docker images` (compare against expected size for the base image alone).
- **Root cause (example):** build tools and source code left in the final image instead of using a multi-stage build (Part 11); an unpinned, full (non-slim) base image (Part 31).
- **Fix:** adopt multi-stage builds, switch to a minimal base image, add a `.dockerignore` (Part 10.2).
- **Prevention:** treat image size as a reviewed metric in CI (fail the build/flag a warning above a size threshold).

27.19 26.19 Build Cache Not Working

- **Symptoms:** every build re-runs every step, even for trivial changes.
- **Commands:** review Dockerfile instruction order (Part 29), `docker build --progress=plain` to see which steps show CACHED vs actually executing.
- **Root cause (example):** `COPY . .` placed **before** the dependency-install step — any source file change (even a comment) invalidates the cache for everything after it, including the expensive dependency install.
- **Fix:** reorder — copy dependency manifests (`requirements.txt/package.json`) and install **before** copying the rest of the source (Part 10.3’s “Good” example).
- **Prevention:** structure every Dockerfile least-to-most-frequently-changing, as a standard team convention.

27.20 26.20 Container Cannot Access the Internet

- **Symptoms:** `apt-get/pip/curl` inside a container times out reaching external hosts.
- **Commands:** `docker exec <container> curl -v https://example.com`, `docker exec <container> cat /etc/resolv.conf`, check host-level networking/DNS (Linux course Part 14.9), check `docker network inspect` for the network’s configuration, check host firewall/NAT rules and any corporate proxy requirements.

- **Root cause (example):** the container is on a custom network with no route out (misconfigured, or intentionally isolated), or the host itself has no outbound connectivity/DNS.
- **Fix:** verify the host has outbound connectivity first (isolate the layer — Linux course Part 14.9's chain applies), then correct the container's network attachment.
- **Prevention:** don't over-isolate networks without a deliberate reason; document which networks are meant to have outbound internet access and which are intentionally internal-only.

28 PART 27 — Docker Debugging Commands

28.1 27.1 Docker-Level Tools

Command	What it reveals
<code>docker logs</code> <code>docker inspect</code>	Application stdout/stderr (Part 24) Full JSON config/state — mounts, network settings, env vars, restart policy, exit code, OOM status
<code>docker exec</code>	Run any diagnostic command inside the container's live namespaces
<code>docker stats</code> <code>docker top</code>	Live resource usage (Part 22.6) Processes running inside a container, viewed from the host (no exec needed — useful if the container has no shell at all, e.g., a scratch-based image)
<code>docker events</code>	Live stream of daemon-level events (container die, OOM, network connect/disconnect) — invaluable for correlating <i>when</i> something happened across multiple containers
<code>docker diff</code>	Filesystem changes vs the image — useful for spotting unexpected writes (or confirming a container hasn't unexpectedly modified something it shouldn't have)
<code>docker cp</code>	Pull a file out of a container for offline inspection (a core dump, a generated config) without needing a shell inside it

```
docker events --filter 'event=die' --filter 'event=oom'    # watch specifically for
↳ crashes/OOM kills, live
docker inspect <container> --format '{{json .State}}' | jq .
```

28.2 27.2 Connecting Back to Linux-Level Tools

Because a container is ultimately just a normal Linux process (Part 1.1, Part 3), every standard Linux diagnostic tool works too — either from **inside** the container (via `docker exec`, if the image has the tool installed) or from the **host**, targeting the container's host-visible PID.

Linux tool	Use inside/around a container
<code>ps</code>	<code>docker exec <c> ps aux</code> — process list inside the container
<code>top/htop</code>	Live resource view from inside, or <code>docker top <c></code> from the host without exec
<code>ss</code>	<code>docker exec <c> ss -tulnp</code> — what's listening inside the container's network namespace
<code>lsof</code>	<code>docker exec <c> lsof -i :5000</code> — what's bound to a port inside the container

Linux tool	Use inside/around a container
df/du	Check disk usage inside the container's filesystem view, separate from docker system df's host-level view
free	Memory as seen inside the container (note: in cgroup v1 setups, free inside a container can misleadingly report the host's total memory rather than the container's cgroup limit — a classic gotcha; cgroup v2 with proper tooling handles this more correctly)
ip	docker exec <c> ip addr — the container's actual network namespace configuration
nslookup/dig	docker exec <c> nslookup backend — verify Docker's embedded DNS resolution directly (Part 16)
curl	The single most useful cross-layer tool — test connectivity from inside one container to another, or to an external service, at the actual application-protocol level

```
# A typical debugging session for "container can't reach the database"
docker exec app-container curl -v http://database:5432           # app-layer view from
↳ inside
docker exec app-container nslookup database                     # does DNS even
↳ resolve?
docker network inspect app-net                                  # are both
↳ containers actually attached?
docker logs database-container --tail 50                       # is the DB
↳ itself healthy?
```

Common mistakes: debugging purely at the Docker level (docker logs, docker inspect) and never dropping into docker exec to check the actual Linux-level state (sockets, DNS, filesystem) from inside the container's own namespaces — many issues (a stale DNS cache, a wrong file permission, a listening-on-wrong-interface bug) are only visible from inside; conversely, debugging purely at the Linux/host level and forgetting to check Docker-specific state (OOMKilled, restart policy, mount configuration) that wouldn't be obvious from host tools alone. Use both layers together, as this part demonstrates.

29 PART 28 — Docker Performance

29.1 28.1 What Affects Performance

Factor	Why it matters
Image size	Slower pulls/pushes, slower deploys, more registry storage cost, larger attack surface
Build time	Slower CI feedback loops, slower iteration
Container startup time	Affects deploy speed, autoscaling responsiveness, rolling-update duration
CPU/Memory allocation	Under-provisioned limits throttle real workloads; over-provisioned limits waste host capacity

Factor	Why it matters
Disk I/O	OverlayFS/copy-on-write overhead on write-heavy workloads; volume driver performance
Network performance	Bridge networking adds a small overhead vs - -network host; matters for very high-throughput services
Layer caching	Directly determines build speed on every iteration (Part 29)

29.2 28.2 Optimization Strategies, by Factor

- **Image size** → multi-stage builds (Part 11), minimal base images (Part 31), `.dockerignore` (Part 10.2), combining RUN layers thoughtfully (Part 10.3).
- **Build time** → efficient cache ordering (Part 29), BuildKit parallel stage execution (Part 30), a shared build cache in CI (e.g., registry-based cache export/import).
- **Startup time** → smaller images pull faster; lazy-initialization patterns in the app; avoid heavy synchronous work before the health check can pass.
- **CPU/Memory** → set limits from real, measured usage (`docker stats` under realistic load), not guesses.
- **Disk I/O** → for write-heavy workloads, consider volume mounts (which bypass some OverlayFS copy-on-write overhead) rather than writing heavily to the container's own writable layer.
- **Network** → - -network host removes NAT/bridge overhead for extreme-throughput cases (at the cost of network isolation — a real trade-off, not a free win).

Common mistakes: optimizing image size aggressively (e.g., jumping straight to scratch/distroless) before the application even works reliably on a normal base — get correctness first, then optimize with multi-stage builds once the build is stable; benchmarking container startup/performance on a laptop and assuming production (different CPU, different I/O characteristics, different network) will behave identically.

30 PART 29 — Docker Build Cache

30.1 29.1 How Caching Works

```
Dockerfile
↓
Instruction    (e.g., RUN pip install -r requirements.txt)
↓
Layer          (the filesystem diff that instruction produced)
↓
Cache          (keyed by: the instruction text itself + the state of everything
↳ before it)
```

For each instruction, Docker checks whether it has already built this **exact** instruction, given the **exact** same preceding layers. If so, it reuses the cached layer instantly instead of re-executing anything. **The moment any instruction's cache is invalidated, every subsequent instruction's cache is invalidated too** — even if those later instructions themselves didn't change at all. This is why instruction order matters enormously.

30.2 29.2 Why Changing One Line Can Invalidate Everything After It

```
FROM python:3.12-slim
COPY . .                                # <- changes EVERY time any file in your repo changes
RUN pip install -r requirements.txt      # <- now re-runs EVERY time too, even if
↳ requirements.txt didn't change!
CMD ["python3", "app.py"]
```

Because `COPY . .` copies your **entire** source tree, changing even a single comment in an unrelated file busts that layer's cache — which then busts **every layer after it**, including the (often slow) dependency install, even though `requirements.txt` itself never changed.

```
FROM python:3.12-slim
COPY requirements.txt .                  # only busts when requirements.txt itself
↳ changes
RUN pip install -r requirements.txt      # stays CACHED across ordinary code changes
COPY . .                                # busts on every code change — but it's
↳ cheap, just a copy
CMD ["python3", "app.py"]
```

Reordering so the **rarely-changing, expensive** step (dependency install) comes before the **frequently-changing, cheap** step (copying source) means most day-to-day builds skip straight to a fast, cached dependency layer.

30.3 29.3 General Structuring Principle

Order Dockerfile instructions from least frequently changing to most frequently changing:

1. FROM (base image) — changes rarely
2. System package installs (RUN apt-get ...) — changes rarely
3. Dependency manifests + install (COPY requirements.txt; RUN pip install) — changes occasionally
4. Application source (COPY . .) — changes constantly
5. Final CMD/ENTRYPOINT config — changes rarely

30.4 29.4 Verifying Cache Behavior

```
docker build --progress=plain -t myapp .      # see exactly which steps show CACHED
↳ vs executing fresh
docker build --no-cache -t myapp .             # force a completely fresh build,
↳ ignoring all cache                           # (useful to rule out a
                                              ↳ stale-cache bug, or for a
                                              # periodic "clean" build in CI to
                                              ↳ catch drift)
```

Common mistakes: `COPY . .` too early (the single most common cache-busting mistake in real Dockerfiles); splitting `apt-get update` and `apt-get install` across separate `RUN` layers (Part 10.3/26.4) — if `update` is cached from days ago but `install` re-runs fresh, you can install packages against a stale index; assuming a `.dockerignore` miss (e.g., forgetting to exclude a frequently-changing build artifact directory) isn't affecting cache — it is, since anything not ignored is part of what `COPY . .` hashes.

31 PART 30 — BuildKit

31.1 30.1 What BuildKit Is

BuildKit is Docker’s modern build engine (default since Docker 23+ for `docker build`, and always used by `docker buildx`), replacing the older, simpler “classic” builder. It’s not just an internal optimization — it exposes genuinely useful new Dockerfile/build features.

31.2 30.2 Key Practical Features

Parallel builds — BuildKit automatically analyzes the dependency graph between build stages (in a multi-stage Dockerfile) and executes **independent stages concurrently**, rather than always strictly sequentially — meaningfully faster builds for Dockerfiles with multiple independent stages.

Build secrets — pass a secret into the build **without** it ending up baked into any layer (solving Part 19.2’s exact problem, at build time):

```
# syntax=docker/dockerfile:1
FROM python:3.12-slim
RUN --mount=type=secret,id=pip_token \
    PIP_TOKEN=$(cat /run/secrets/pip_token) pip install --index-url \
    ↪ https://$PIP_TOKEN@internal-pypi/ myprivatepkg
```

```
docker build --secret id=pip_token,src=./pip_token.txt -t myapp .
```

The secret is available **only during that specific RUN**, mounted at a temp path, and is **never** written into any committed layer — a real, structural fix, not just a convention to remember.

Build cache mounts — persist a cache directory (like a package manager’s download cache) **across separate builds**, without that cache ever becoming part of the image itself:

```
RUN --mount=type=cache,target=/root/.cache/pip \
    pip install -r requirements.txt
```

This can dramatically speed up repeated builds (e.g., in CI) by reusing previously-downloaded packages, while keeping the final image clean of that cache data.

Better build contexts — BuildKit supports named, multiple build contexts (e.g., referencing a separate local directory or even a different image as an additional source for `COPY --from=`), useful for more complex, modular build setups.

31.3 30.3 Using BuildKit

```
docker buildx build -t myapp .           # buildx is BuildKit-native by default
DOCKER_BUILDKIT=1 docker build -t myapp . # (legacy explicit opt-in; unnecessary on
↪ modern Docker where                    # BuildKit is already the default builder)
```

Common mistakes: writing a Dockerfile that uses BuildKit-only syntax (`--mount=type=secret`, etc.) without the `# syntax=docker/dockerfile:1` directive at the top, or on an environment where BuildKit isn’t active — it will fail or silently behave differently; still manually baking secrets into ARG/ENV (Part 19.2) out of habit when BuildKit’s `--mount=type=secret` solves the exact same problem properly.

32 PART 31 — Container Image Optimization: Choosing a Base Image

32.1 31.1 The Options, Compared

Base	Approx. size	libc	Package manager	Debugging ease	Notes
Ubuntu	~80MB+	glibc	apt	Easy — full toolset available	Familiar, well-supported, larger than necessary for many production images A lighter Debian variant; good default balance of size vs compatibility Very small, but real compatibility risk (below) Minimal attack surface; runtime-only, no OS utilities Only viable for fully static binaries (e.g., a CGO_ENABLED=0 Go build)
Debian slim	~50-80MB	glibc	apt	Easy	
Alpine	~5-8MB	musl (not glibc!)	apk	Harder — minimal tools by default	
Distroless	~20-30MB (varies)	glibc (Debian-based distroless)	none — no shell, no package manager at all	Hard — no shell to exec into at all	
Scratch	~0MB (literally empty)	none	none	Hardest — nothing but your binary	

32.2 31.2 Security, Compatibility, Debugging, Size — The Real Trade-offs

- **Security:** smaller images generally mean fewer installed packages, meaning fewer potential CVEs and a smaller attack surface — distroless/scratch are the strongest here (no shell for an attacker to even get an interactive foothold with, in the event of code execution).
- **Compatibility:** **Alpine's biggest practical risk** — it uses **musl libc**, not the **glibc** most software (especially prebuilt Python/Node native extensions, some Java JVM behaviors) is built and tested against. This causes real, sometimes subtle bugs: DNS resolution differences, floating-point behavior differences, native-extension packages that don't have musl-compatible prebuilt wheels (forcing a slow from-source compile, or outright failing).
- **Debugging:** Ubuntu/Debian-slim let you `docker exec ... bash` and use full familiar tooling immediately. Alpine requires learning apk and BusyBox-flavored utilities. Dis-

troless/scratch have **no shell at all** — you cannot `docker exec` into an interactive shell; debugging requires attaching a separate “debug” sidecar/ephemeral container (`docker debug`, or Kubernetes’ `kubectl debug` equivalent) or relying entirely on external logs/metrics.

- **Image size:** Alpine and distroless/scratch win decisively here — often an order of magnitude smaller than Ubuntu.

32.3 31.3 When NOT to Use Alpine

- Your application depends on **Python/Node packages with native C extensions** that don’t ship musl-compatible prebuilt binaries (common enough to be a real, recurring pain point — e.g., some `numpy/pandas/cryptography`-adjacent packages historically).
- You need **exact glibc-level behavior parity** with your production-like dev/CI environment, and subtle musl differences (locale handling, DNS resolution nuances) have caused or could cause real bugs.
- Your team needs to **frequently exec into containers for interactive debugging** and doesn’t want the added friction of relearning `apk/BusyBox` tooling.
- The size savings genuinely don’t matter for your context (e.g., a low-traffic internal tool where a 200MB vs 20MB image makes no practical difference) — in that case, the *compatibility risk* of Alpine may not be worth taking on at all.

32.4 31.4 A Practical Recommendation

- **Default choice for most application images:** `debian-slim`-based official language images (e.g., `python:3.12-slim`, `node:20-slim`) — a strong balance of size, compatibility, and debuggability.
- **When size is critical and you’ve verified compatibility:** Alpine, with real testing against musl (not just “it built”).
- **When maximum security/minimal attack surface matters and you don’t need shell access for debugging:** distroless, especially for statically-compiled languages (Go) or JVM-based apps where the runtime itself is bundled.
- **When you have a truly static binary:** scratch — the smallest and most secure option there is, when it’s viable at all (Part 11.2’s Go example).

Common mistakes: switching to Alpine purely to shrink image size, without testing that every dependency actually works correctly under musl — discovering a subtle bug in production days or weeks later; choosing distroless/scratch for an application the team still needs to interactively debug often, without first establishing an alternative debugging workflow (sidecar containers, robust centralized logging) to compensate for the lack of a shell.

33 PART 32 — Docker with Python

33.1 32.1 The Chain

Python Flask/FastAPI	(your application code)
↓	
Dockerfile	(packages it with its exact dependencies)
↓	
Image	(a built, versioned, portable artifact)
↓	
Container	(a running instance, in dev, CI, or production)

33.2 32.2 Virtual Environments vs Containers

A Python **virtual environment** (venv) isolates Python **package dependencies** from the system Python and other projects on the **same machine** — it does nothing for OS-level dependencies, the Python interpreter version itself, or portability across machines. A **container** isolates the **entire runtime environment** — the exact Python interpreter version, OS-level libraries, and application dependencies — as one portable unit. In practice: **you generally don't need a venv inside a container at all** — the container itself already provides equivalent-or-stronger isolation; installing directly into the container's Python environment (with `--no-cache-dir` to avoid bloating the image with pip's download cache) is the standard, simpler approach.

33.3 32.3 requirements.txt and Reproducibility

```
flask==3.0.3
gunicorn==22.0.0
psycpg2-binary==2.9.9
redis==5.0.7
```

Pin exact versions (`==`, not `>=`) for reproducible builds (Part 10.1) — an unpinned flask could resolve to a different, potentially breaking version on a rebuild months later.

33.4 32.4 A Production Flask/Gunicorn Dockerfile

```
FROM python:3.12-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

RUN useradd -m appuser
USER appuser

EXPOSE 8000
HEALTHCHECK --interval=30s --timeout=3s CMD curl -f http://localhost:8000/health ||
    ↪ exit 1

CMD ["gunicorn", "--bind", "0.0.0.0:8000", "--workers", "4", "app:app"]
```

Why Gunicorn, not Flask's built-in dev server: Flask's built-in `app.run()` server is single-threaded, unoptimized, and explicitly documented as unsuitable for production. **Gunicorn** (a proper WSGI HTTP server) handles multiple worker processes, proper request concurrency, and production-grade stability — using Flask's dev server in a production container image is a common and important beginner mistake to avoid.

33.5 32.5 Environment Variables and the Health Endpoint

```
import os
from flask import Flask, jsonify

app = Flask(__name__)
```

```
@app.route("/health")
def health():
    return jsonify(status="healthy"), 200

if __name__ == "__main__":
    port = int(os.environ.get("PORT", 8000))
```

Config (database URLs, feature flags, the port to bind) should come from environment variables (Part 19), never hardcoded — the same image then runs correctly in dev, staging, and production, differing only in the environment variables/secrets supplied at docker run/Compose/Kubernetes deploy time. This is exactly what makes the image itself immutable (Part 1.9) while still being environment-flexible.

Common mistakes: shipping Flask’s dev server (`app.run(debug=True)`) in a production image — both a performance problem and, with `debug=True`, a serious security issue (it can expose an interactive debugger console); forgetting `--no-cache-dir` on `pip install`, leaving pip’s download cache baked into the image for no benefit; not pinning dependency versions, causing “worked in CI last week, fails today” drift.

34 PART 33 — Docker with Nginx

34.1 33.1 The Pattern

```
Internet
↓
Nginx container    (reverse proxy – the ONLY container with a published port)
↓
Application container(s)  (internal-only, reached by Nginx via container/service name)
```

34.2 33.2 Reverse Proxy Configuration

```
# nginx.conf
server {
    listen 80;

    location / {
        proxy_pass http://backend:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

```
services:
  nginx:
    image: nginx:1.27
    ports: ["80:80"]
    volumes:
      - ./nginx.conf:/etc/nginx/conf.d/default.conf:ro
    depends_on: [backend]
    networks: [app-net]

  backend:
    build: ./backend
    networks: [app-net]    # NOTE: no `ports:` here – not reachable directly from
    ↪ outside the host
```

Nginx forwards to backend **by container/service name** (Part 14.3, Part 16.2) — the actual container IP is irrelevant and can change freely.

34.3 33.3 Serving Static Files

```
server {
    listen 80;

    location /static/ {
        alias /usr/share/nginx/html/static;
        expires 30d;
    }

    location / {
        proxy_pass http://backend:8000;
    }
}
```

```
}  
}
```

```
FROM nginx:1.27  
COPY ./static /usr/share/nginx/html/static  
COPY nginx.conf /etc/nginx/conf.d/default.conf
```

Serving static assets (JS bundles, images, CSS) directly from Nginx — rather than proxying every request through to the application server — is significantly more efficient: Nginx is purpose-built and highly optimized for exactly this.

34.4 33.4 SSL/TLS Termination

```
server {  
    listen 443 ssl;  
    ssl_certificate /etc/nginx/certs/fullchain.pem;  
    ssl_certificate_key /etc/nginx/certs/privkey.pem;  
  
    location / {  
        proxy_pass http://backend:8000;  
        proxy_set_header X-Forwarded-Proto https;  
    }  
}  
  
server {  
    listen 80;  
    return 301 https://$host$request_uri; # redirect all plain HTTP to HTTPS  
}
```

“**TLS termination**” means Nginx handles the encrypted connection with the client, then talks to the backend over **plain HTTP** internally (safe, since that traffic never leaves the isolated Docker network) — the backend application itself doesn’t need to know anything about certificates at all. In many production AWS setups, TLS termination happens even earlier, at an **Application Load Balancer** (Part 35), with Nginx (if used at all) handling only internal routing.

34.5 33.5 Load Balancing

```
upstream backend_pool {  
    server backend1:8000;  
    server backend2:8000;  
    server backend3:8000;  
}  
  
server {  
    listen 80;  
    location / {  
        proxy_pass http://backend_pool;  
    }  
}
```

```
services:  
  backend:  
    build: ./backend  
    deploy:  
      replicas: 3 # NOTE: `deploy.replicas` is a Swarm/Compose-in-Swarm-mode  
↪ concept —
```

```

↪ container per                                # a plain `docker compose up` on a single host runs one
↪ up --scale backend=3`                        # service unless scaled explicitly with `docker compose
networks: [app-net]

```

For genuine multi-host load balancing and scaling in production, this responsibility typically shifts to **Kubernetes Services** (Part 37) or an **AWS ALB in front of ECS/EKS** (Part 35) rather than a single Nginx container’s upstream block — but the underlying reverse-proxy concept is identical.

Common mistakes: terminating TLS at the application server itself instead of Nginx/a load balancer, adding unnecessary complexity and certificate-management burden to every application; forgetting `proxy_set_header X-Forwarded-For/X-Forwarded-Proto`, causing the backend application to see every request as coming from Nginx’s internal IP over plain HTTP, breaking client-IP-based logic (rate limiting, geolocation) and “is this request secure” checks.

35 PART 34 — Docker + CI/CD (Jenkins)

35.1 34.1 The Full Pipeline

```

GitHub                (developer pushes code)
↓
Jenkins                (triggered by webhook/poll)
↓
Tests                  (unit/integration tests run BEFORE building an image – fail fast)
↓
Docker Build           (produces a tagged, versioned image)
↓
Security Scan          (Trivy – Part 21.8 – fail the build on critical CVEs)
↓
Docker Push            (to ECR/Docker Hub/private registry – Part 20)
↓
Registry               (the image now exists as a deployable artifact)
↓
Deployment              (to EC2, ECS/Fargate, or Kubernetes – Parts 35, 37)

```

35.2 34.2 A Complete Jenkinsfile

```

pipeline {
    agent any

    environment {
        IMAGE_NAME = "myregistry/myapp"
        IMAGE_TAG  = "${env.BUILD_NUMBER}"
        ECR_REGISTRY = "123456789012.dkr.ecr.us-east-1.amazonaws.com"
    }

    stages {
        stage('Checkout') {
            steps {
                checkout scm
            }
        }
    }
}

```

```

stage('Run Tests') {
    steps {
        sh 'pip install -r requirements.txt'
        sh 'pytest tests/'
    }
}

stage('Docker Build') {
    steps {
        sh "docker build -t ${IMAGE_NAME}:${IMAGE_TAG} ."
    }
}

stage('Security Scan') {
    steps {
        sh "trivy image --exit-code 1 --severity CRITICAL,HIGH
↪  ${IMAGE_NAME}:${IMAGE_TAG}"
    }
}

stage('Push to ECR') {
    steps {
        sh """
            aws ecr get-login-password --region us-east-1 | \
            docker login --username AWS --password-stdin ${ECR_REGISTRY}
            docker tag ${IMAGE_NAME}:${IMAGE_TAG}
↪  ${ECR_REGISTRY}/myapp:${IMAGE_TAG}
            docker push ${ECR_REGISTRY}/myapp:${IMAGE_TAG}
        """
    }
}

stage('Deploy') {
    steps {
        sh "aws ecs update-service --cluster prod --service myapp
↪  --force-new-deployment"
    }
}

post {
    failure {
        echo 'Pipeline failed – check the stage logs above.'
    }
    always {
        sh "docker rmi ${IMAGE_NAME}:${IMAGE_TAG} || true" // clean up the local
↪  build agent's image
    }
}
}

```

35.3 34.3 Why Tests and Scans Happen *Before* Push

Running tests and the security scan **before** docker push means a broken or vulnerable image **never reaches the registry at all** — nothing downstream (staging, production, another team pulling the image) can accidentally consume it. The `trivy image --exit-code 1` flag makes

the scan a genuine **release gate**: the pipeline stage fails, and Jenkins stops, if critical/high CVEs are found — this is the practical mechanism, not just a report nobody reads.

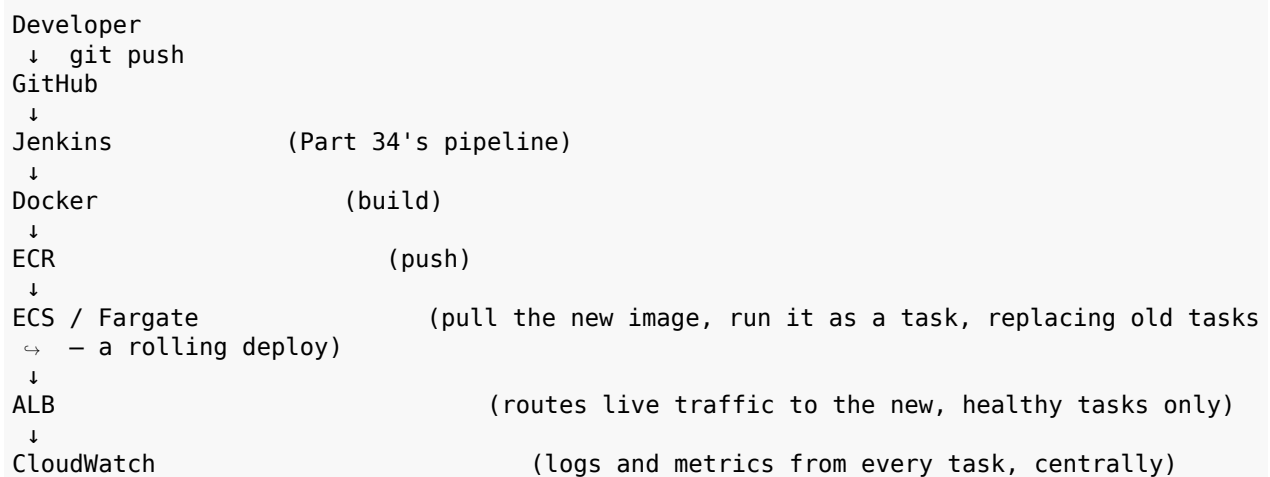
Common mistakes: running the security scan or tests **after** the push (too late — the vulnerable/broken image is already distributable); building the Docker image on the same host/agent as production without any isolation, letting a malicious Dockerfile/build script potentially affect the CI infrastructure itself (mitigated by using ephemeral, isolated build agents); not cleaning up images/containers on long-lived Jenkins build agents, slowly filling their disk over hundreds of builds (Part 26.11's disk-full scenario, CI-agent edition).

36 PART 35 — Docker + AWS

36.1 35.1 The Building Blocks

AWS service	Role
EC2 + Docker	Run Docker directly on a self-managed EC2 instance — full control, more operational overhead
ECR (Elastic Container Registry)	AWS's native image registry (Part 20.1) — IAM-authenticated, tightly integrated with ECS/EKS
ECS (Elastic Container Service)	AWS's own container orchestrator — runs containers as “tasks” on a “cluster,” without needing Kubernetes
Fargate	A serverless compute mode for ECS (and EKS) — no EC2 instances to manage at all; you specify CPU/memory per task and AWS runs it
ALB (Application Load Balancer)	Distributes incoming traffic across running tasks/containers, handles TLS termination, health-check-based routing
CloudWatch	Metrics and log aggregation (Part 25.2, Linux course Part 27.4)
IAM	Controls what an ECS task (or EC2 instance) is allowed to do via AWS APIs — no static credentials needed inside the container (Linux course Part 27.2)
Secrets Manager	Injects secrets into ECS tasks at runtime without baking them into images or task definitions in plaintext (Part 19.3)

36.2 35.2 The Full Flow



36.3 35.3 EC2 + Docker (Self-Managed)

```
# On an EC2 instance (Linux course Part 27)
sudo apt install -y docker.io
sudo systemctl enable --now docker
aws ecr get-login-password --region us-east-1 | docker login --username AWS
  ↪ --password-stdin <account>.dkr.ecr.us-east-1.amazonaws.com
docker pull <account>.dkr.ecr.us-east-1.amazonaws.com/myapp:1.4.2
docker run -d --restart unless-stopped -p 80:8000
  ↪ <account>.dkr.ecr.us-east-1.amazonaws.com/myapp:1.4.2
```

This is the simplest possible AWS deployment model — you own the instance, the Docker daemon, and every update, exactly like the Linux course’s capstone project (Linux course Part 34). It doesn’t scale or self-heal automatically the way ECS/Fargate does.

36.4 35.4 ECS/Fargate (Managed)

An ECS **task definition** describes, in JSON, exactly what a plain `docker run` command would need — image, port mappings, environment variables/secrets, CPU/memory limits, IAM role — but AWS itself handles actually scheduling and running it (on EC2 instances you manage, or on **Fargate**, where AWS manages the underlying compute entirely).

```
{
  "family": "myapp",
  "networkMode": "awsvpc",
  "requiresCompatibilities": ["FARGATE"],
  "cpu": "512",
  "memory": "1024",
  "containerDefinitions": [
    {
      "name": "myapp",
      "image": "123456789012.dkr.ecr.us-east-1.amazonaws.com/myapp:1.4.2",
      "portMappings": [{"containerPort": 8000}],
      "secrets": [
        {
          "name": "DB_PASSWORD", "valueFrom":
            ↪ "arn:aws:secretsmanager:us-east-1:123456789012:secret:db-password"
        }
      ],
      "logConfiguration": {
        "logDriver": "awslogs",
        "options": {
          "awslogs-group": "/ecs/myapp",
          "awslogs-region": "us-east-1",
          "awslogs-stream-prefix": "myapp"
        }
      }
    }
  ]
}
```

Notice the direct parallels to everything covered so far: image (Part 8), portMappings (Part 15), secrets referencing Secrets Manager (Part 19.3), logConfiguration using the `awslogs` driver (Part 24.2’s logging-driver concept, AWS-flavored), cpu/memory (Part 22) — ECS task definitions are essentially a structured, managed equivalent of everything a `docker run`/Compose file already expresses.

36.5 35.5 Docker and AWS EKS (Brief Preview of Part 37)

EKS (Elastic Kubernetes Service) is AWS’s **managed Kubernetes** offering — the same containers, built the exact same way with Docker, but orchestrated by Kubernetes instead of ECS. Whether you choose ECS or EKS is an orchestration-layer decision; the Docker build/image/registry story (Parts 4–20) is **identical either way** — this is exactly the point of OCI standardization (Part 39).

Common mistakes: choosing EC2+Docker for a workload that genuinely needs autoscaling/self-healing (reinventing what ECS/EKS already provide, poorly, by hand); hardcoding AWS credentials inside a container instead of using an IAM role attached to the EC2 instance or ECS task (Linux course Part 27.2) — the same “never bake secrets/credentials into images” principle from Part 19, applied specifically to AWS API access.

37 PART 36 — Docker + Terraform

37.1 36.1 Different Responsibilities, Working Together

Docker

→ Builds the APPLICATION CONTAINER (what runs)

Terraform

→ Creates the INFRASTRUCTURE (where and how it runs)

Docker answers “what is my application, packaged how?” Terraform answers “what AWS resources does this need to actually run, be reachable, and scale?” They are complementary, not overlapping — a common point of confusion for beginners is trying to make one tool do the other’s job.

37.2 36.2 What Terraform Provisions for a Container Workload

AWS

↓

ECR (the registry Docker images get pushed to)

↓

ECS (cluster + service + task definition) (where containers actually run)

↓

ALB (routes traffic to the running tasks)

↓

Networking (VPC, subnets, Security Groups – everything from the Linux

↔ course's Part 14/27)

37.3 36.3 A Minimal Terraform Example

```
resource "aws_ecr_repository" "myapp" {
  name = "myapp"
}

resource "aws_ecs_cluster" "main" {
  name = "prod-cluster"
}

resource "aws_ecs_task_definition" "myapp" {
  family              = "myapp"
  requires_compatibilities = ["FARGATE"]
  network_mode        = "awsvpc"
  cpu                  = "512"
  memory               = "1024"
  execution_role_arn   = aws_iam_role.ecs_execution.arn

  container_definitions = jsonencode([
    {
      name       = "myapp"
      image      = "${aws_ecr_repository.myapp.repository_url}:latest"
      portMappings = [{ containerPort = 8000 }]
    }
  ])
}
```

```

resource "aws_ecs_service" "myapp" {
  name           = "myapp"
  cluster        = aws_ecs_cluster.main.id
  task_definition = aws_ecs_task_definition.myapp.arn
  desired_count  = 3
  launch_type    = "FARGATE"

  network_configuration {
    subnets      = var.private_subnet_ids
    security_groups = [aws_security_group.myapp.id]
  }

  load_balancer {
    target_group_arn = aws_lb_target_group.myapp.arn
    container_name   = "myapp"
    container_port    = 8000
  }
}

```

```

terraform init
terraform plan      # review exactly what will be created/changed BEFORE applying
terraform apply

```

37.4 36.4 The Boundary in Practice

- **CI/CD (Jenkins, Part 34)** builds the Docker image and pushes it to the ECR repository **Terraform created**.
- **Terraform** does **not** rebuild or repush the image on every terraform apply — it typically references a tag (or is deliberately updated by CI to point at a new tag) and manages the surrounding infrastructure's lifecycle.
- A common, effective pattern: Terraform provisions the ECS service pointing at whatever's currently in ECR; a separate, lightweight deploy step (`aws ecs update-service --force-new-deployment`, as in Part 34's Jenkinsfile) triggers ECS to pull and roll out a newly-pushed image, **without** needing a full terraform apply for every application deployment — infrastructure changes and application deployments are different cadences and different tools' jobs.

Common mistakes: trying to make Terraform “build” the Docker image (not its job — that's Docker/CI's responsibility); running a full terraform apply as your application deployment mechanism for every code change — slow, and conflates infrastructure changes (rare, reviewed carefully) with application deploys (frequent, should be fast); hardcoding an image tag like `:latest` in Terraform, making it unclear which actual image version is running at any given time (prefer explicit, CI-supplied version tags — Part 8.2).

38 PART 37 — Docker + Kubernetes

38.1 37.1 The Relationship, Explicitly

```

Dockerfile
↓ docker build
Docker Image
↓ docker push
Container Registry      (Docker Hub / ECR / etc. — Part 20)

```

```

↓ Kubernetes pulls from here
Kubernetes
↓ schedules onto a node
Pod
↓ runs
Container

```

Everything from Parts 1–33 of this course — the image, the Dockerfile, the registry — is **identical** whether the container ends up running under plain Docker, ECS, or Kubernetes. Kubernetes’ job starts **after** the image exists: scheduling, networking, scaling, and self-healing many containers across many machines.

38.2 37.2 Why Kubernetes Needs Container Images

Kubernetes doesn’t define its own container/image format — it consumes the **exact same OCI-compliant images** (Part 39) this whole course has been building. A Kubernetes **Pod** spec references an image by name/tag/digest, exactly like a `docker run` command would — Kubernetes then handles pulling it, and instructs the node’s container runtime to actually run it.

38.3 37.3 OCI, containerd, CRI, runc — How Kubernetes Actually Runs a Container

```

kubelet (the Kubernetes agent running on every node – Linux course Part 29.2)
↓ (via CRI – Container Runtime Interface, a standardized API)
containerd (or another CRI-compliant runtime – Part 2, Part 4.2)
↓
runc (creates the actual namespaces/cgroups – Part 3)
↓
Linux Kernel

```

- **CRI (Container Runtime Interface):** a standardized API Kubernetes uses to talk to **any** compliant container runtime — this is precisely what decouples Kubernetes from needing “Docker” specifically.
- **containerd:** the most common CRI-compliant runtime in modern Kubernetes clusters (including EKS) — notably, this is the **exact same containerd** that sits underneath the Docker daemon itself (Part 4.2)! Kubernetes just talks to it directly, without dockerd in the middle.
- **runc:** the low-level, OCI-spec-compliant component that actually creates the container (namespaces, cgroups) — used by containerd regardless of whether Docker or Kubernetes is on top.

38.4 37.4 Docker vs Kubernetes — Clearing Up the Confusion

	Docker	Kubernetes
What it is	A tool for building and running individual containers on one host	An orchestrator for running many containers across many hosts
Builds images?	Yes (<code>docker build</code>)	No — Kubernetes never builds images; that’s still Docker’s (or another OCI-compliant builder’s) job

	Docker	Kubernetes
Runs containers?	Yes, directly	Yes, but via a CRI-compliant runtime (typically containerd) — not necessarily “Docker” at all
Scheduling across machines?	No — a single Docker host only	Yes — this is Kubernetes’ core purpose
Self-healing/scaling?	No (Swarm adds some of this — Part 38)	Yes, natively

38.5 37.5 The Historical “Docker vs Kubernetes” Relationship

Early Kubernetes versions used **Docker itself** (via a component called `dockershim`) as their container runtime — Kubernetes literally called the `docker` CLI/API under the hood. As Kubernetes matured, the community standardized on the **CRI** to decouple from any single runtime implementation, and Kubernetes **removed `dockershim` entirely** (as of Kubernetes 1.24, in 2022) in favor of talking to CRI-compliant runtimes like `containerd` **directly**. This does **not** mean “Docker died” or that Docker-built images stopped working — it means Kubernetes stopped needing the full `dockerd` daemon specifically; **images you build with docker build work exactly the same as before**, because they’re standard OCI images (Part 39), and `containerd` (which Docker itself is built on) runs them either way.

Common mistakes: believing “Kubernetes doesn’t support Docker anymore” means Docker-built images are somehow incompatible with Kubernetes — they are not; the *build tool* (Docker) and the *image format it produces* (OCI) are separate concerns from the *runtime* Kubernetes uses to execute that image, and only the runtime detail changed.

39 PART 38 — Docker Swarm (Brief)

39.1 38.1 What It Is

Docker Swarm is Docker's own **native** orchestrator — built directly into the Docker Engine (`docker swarm init`), letting you run and scale containers as **services** across a **cluster of Docker hosts (nodes)**, with built-in load balancing and a declarative desired-state model, using the same Compose file format (Part 17) with a `deploy` section.

39.2 38.2 Architecture

```
Manager Node(s)    - maintain cluster state, schedule services, handle the Raft
  ↳ consensus for HA
  ↓
Worker Node(s)      - actually run the containers ("tasks") assigned by managers
```

```
docker swarm init                                # turn the current host into a manager
docker swarm join --token <token> <manager-ip>:2377 # join a worker to the swarm
docker service create --replicas 3 -p 80:80 nginx    # run 3 replicas,
  ↳ load-balanced automatically
docker service scale nginx=5                        # scale up/down declaratively
```

Swarm creates an **overlay network** (Part 14.1) automatically spanning every node, so containers on different physical hosts can still reach each other by service name — the same DNS-based service discovery philosophy (Part 16), just extended across a cluster.

39.3 38.3 Why Kubernetes Became More Widely Used

- **Ecosystem and extensibility:** Kubernetes has a vastly larger ecosystem (Helm, Operators, custom controllers, service meshes) and a pluggable architecture (CRI, CNI, CSI) that Swarm's simpler, more opinionated model doesn't match.
- **Feature depth:** Kubernetes offers far more sophisticated scheduling (affinity/anti-affinity, taints/tolerations), autoscaling (HPA/VPA/Cluster Autoscaler), and configuration management (ConfigMaps, Secrets, extensive RBAC) out of the box.
- **Industry/cloud adoption:** every major cloud provider offers a managed Kubernetes service (EKS, GKE, AKS) with deep integration; managed Swarm offerings never reached comparable adoption.
- **Swarm's advantage, where it still applies:** genuinely **much simpler** to set up and operate for smaller, less complex deployments — if a team's needs are modest and they're already fully in the Docker CLI/Compose ecosystem, Swarm remains a legitimate, lower-overhead choice. This course does not recommend investing deep, extended effort mastering Swarm given Kubernetes' dominance in most production DevOps environments — know it exists and roughly how it works, then focus your depth on Kubernetes (Part 37, and the standalone Kubernetes-focused material it points toward).

40 PART 39 — OCI (Open Container Initiative)

40.1 39.1 Why It Exists

Before OCI, “a container” was effectively defined by whatever Docker’s internal implementation happened to do — there was no vendor-neutral standard. The **Open Container Initiative** (founded 2015, under the Linux Foundation) standardized the two things that actually needed to be portable across tools/vendors:

- **OCI Image Specification:** defines exactly what a container image **is** — its layer format, manifest structure, configuration format. Any OCI-compliant tool can build, inspect, and run any OCI-compliant image, regardless of which tool built it.
- **OCI Runtime Specification:** defines exactly how a compliant runtime should **execute** a container, given an unpacked filesystem bundle and a configuration file — this is the spec runc (Part 2, Part 37.3) implements.

40.2 39.2 Practical Implications

```
Image built by: docker build, buildah, kaniko, Cloud Native Buildpacks...
↓ (all produce OCI-spec-compliant images)
Runtime executed by: runc, crun, gVisor, Kata Containers...
↓ (all consume OCI-spec-compliant images/bundles)
```

Because of this standardization, an image built with `docker build` runs correctly under `containerd+runc` on a Kubernetes node with **zero** Docker-specific dependency — this is the technical foundation underneath Part 37.5’s “Docker images still work fine even though Kubernetes stopped using `dockerd` directly.”

```
docker inspect --format='{{.RootFS}}' myimage      # OCI-standardized layer/filesystem
↪ metadata under the hood
```

40.3 39.3 Why Containers Are Not Limited to Docker

Docker remains the most popular **build and local-development tool**, but the actual container ecosystem includes many OCI-compliant alternatives at every layer: - **Build:** Docker, buildah, kaniko (build images without needing a Docker daemon at all — common in Kubernetes-native CI), Cloud Native Buildpacks. - **Runtime:** runc (standard), crun (a faster, lower-memory C implementation), gVisor/Kata Containers (stronger-isolation runtimes for higher-security multi-tenant needs — Part 3.4’s kernel-sharing concern, addressed). - **Registry:** Docker Hub, ECR, GCR, Harbor, and others — all speak the same standardized **OCI Distribution Specification** for pushing/pulling images.

Common mistakes: conflating “Docker” (a specific company/tool) with “containers” (the broader, standardized technology) — this course has deliberately used Docker as the vehicle to teach containers, but the underlying skills (namespaces, cgroups, OCI images, Dockerfile-equivalent build patterns) transfer directly to any OCI-compliant tool you encounter.

41 PART 40 — Container Security & Supply Chain

41.1 40.1 The Supply-Chain Question

“Is this container image actually safe to run?” is really several distinct questions: *where did the base image come from and is it trustworthy? What dependencies does it contain, and are any known-vulnerable? Has the image been tampered with since it was built? Can we prove/trace exactly how and from what source this image was produced?*

41.2 40.2 Base Image Security and Dependency Vulnerabilities

- Prefer **official, actively-maintained base images** (Docker Official Images, verified publisher images) over random, unmaintained third-party images.
- **Pin versions** (Part 8.2, Part 10.1) — an unpinned base can silently introduce new vulnerabilities (or remove old ones) between builds without any visibility.
- **Scan regularly, not just once** — a base image that was clean at build time can have a **newly disclosed** CVE affect it later; scanning should be a recurring/scheduled process against images already in production, not solely a pre-deploy gate (Part 21.8, Part 34.3).

41.3 40.3 SBOM (Software Bill of Materials)

An **SBOM** is a complete, structured inventory of everything inside an image — every package, every version, every dependency (including transitive ones) — analogous to an ingredients list. It enables fast, precise answers to “are we affected by CVE-XXXX-YYYY” across your entire fleet of images, without re-scanning everything from scratch each time a new CVE is disclosed.

```
trivy image --format cyclonedx --output sbom.json myapp:1.0      # generate an SBOM in a
  ↳ standard format
syft myapp:1.0 -o spdx-json                                     # another common
  ↳ SBOM-generation tool
```

41.4 40.4 Image Signing and Provenance (Cosign)

Image signing cryptographically proves an image hasn’t been tampered with since it was built by a trusted party — solving “how do I know this image in the registry is really the one my CI pipeline built, and not something an attacker swapped in?”

```
cosign generate-key-pair                                         # generate a signing keypair
cosign sign --key cosign.key myregistry/myapp:1.0              # sign the image after
  ↳ building/pushing it
cosign verify --key cosign.pub myregistry/myapp:1.0            # verify before deploying – fail
  ↳ the deploy if unsigned/invalid
```

Provenance goes a step further — an attestation of **how** the image was built (which source commit, which CI pipeline, which build steps) — enabling verification not just of “was this signed by someone trusted” but “was this specifically built by our real pipeline, from our real source repository.”

41.5 40.5 SLSA (Supply-chain Levels for Software Artifacts)

SLSA is a framework (originated at Google, now community-governed) defining **maturity levels** for supply-chain security practices — from basic (SLSA 1: builds are scripted/documented) up to SLSA 4 (fully verifiable, hermetic, tamper-resistant builds with strong provenance). It’s a

useful **shared vocabulary and checklist**, not a tool itself — many CI/CD and registry ecosystems (including GitHub Actions, GitLab) are building increasingly direct SLSA-level attestation support.

41.6 40.6 Secrets, Least Privilege, Rootless Containers (Recap and Extension)

- **Secrets:** never baked into images (Part 19.2); use file-mounted secrets or a real secrets manager (Part 19.3) as a matter of supply-chain hygiene, not just runtime security — a leaked image is a leaked secret if it was ever baked in, permanently, in every historical tag.
- **Least privilege:** non-root users (Part 21.1), dropped capabilities (Part 21.3), read-only filesystems (Part 21.5) — every one of these reduces what a supply-chain compromise (a malicious dependency, a compromised base image) can actually **do** even if it does get executed.
- **Rootless containers:** beyond just running the *container's process* as non-root (Part 21.1), rootless mode runs the **Docker daemon itself** as a non-root user on the host — meaning even a full daemon-level compromise doesn't automatically grant host root. A meaningfully stronger security posture for high-sensitivity or genuinely multi-tenant hosts, at the cost of some operational complexity/feature limitations.

41.7 40.7 A Practical DevSecOps Checklist

1. Use official/verified, pinned base images.
2. Generate and retain an SBOM for every build.
3. Scan every image in CI (Trivy) — fail the build on critical/high CVEs.
4. Re-scan images already running in production on a recurring schedule.
5. Sign every image after build (Cosign); verify signatures before deploy.
6. Never bake secrets into any layer — use a real secrets manager.
7. Run as non-root; drop unnecessary capabilities; prefer read-only filesystems.
8. Track provenance — know which commit/pipeline/build produced every running image.

Common mistakes: scanning only at build time and never again — a clean image today can be vulnerable next week as new CVEs are disclosed against its exact same dependency versions; treating image signing as “too advanced/unnecessary” for a team's current maturity level without weighing the real risk of an unverified/tampered image reaching production, especially in any environment with untrusted or loosely-controlled registry access.

42 PART 41 — Production Best Practices Checklist

A consolidated, print-and-check reference pulling together every part of this course.

42.1 41.1 Image

- ☐ Small, minimal base image appropriate for the workload (Part 31).
- ☐ Multi-stage build used where compilation/build tooling isn't needed at runtime (Part 11).
- ☐ `.dockerignore` in place (Part 10.2).
- ☐ Versions pinned — base image, OS packages, language dependencies (Part 8.2, Part 10.1).
- ☐ Dockerfile instruction order optimized for build cache (Part 29).
- ☐ Image scanned for vulnerabilities in CI, with a real fail threshold (Part 21.8, Part 34.3).
- ☐ SBOM generated and retained per build (Part 40.3).
- ☐ Image signed after build; signature verified before deploy (Part 40.4).

42.2 41.2 Container Configuration

- ☐ Runs as a **non-root** user (Part 21.1).
- ☐ Unnecessary Linux capabilities dropped (Part 21.3).
- ☐ Read-only root filesystem where feasible, with explicit writable mounts for what actually needs it (Part 21.5).
- ☐ `--privileged` **never** used for application containers (Part 21.6).
- ☐ Resource limits (`--cpus`, `--memory`, `--pids-limit`) set from real, measured usage (Part 22).
- ☐ HEALTHCHECK/readiness endpoint defined and genuinely reflects service health (Part 23).
- ☐ Entrypoint correctly forwards signals — `exec'd`, not left running under an unhandled shell wrapper (Part 12.2).
- ☐ Appropriate restart policy set (unless `- stopped` for most long-running production services) (Part 12.4).

42.3 41.3 Secrets

- ☐ No secrets baked into any image layer, ever (Part 19.2).
- ☐ Real secrets use a secrets manager (AWS Secrets Manager/SSM) or file-mounted Docker secrets — not plain env vars, for anything genuinely sensitive (Part 19.3).
- ☐ `.env` files with real values are `.gitignore'd`, never committed (Part 19.4).

42.4 41.4 Networking

- ☐ Custom networks used for multi-container/service communication, never the default bridge (Part 14.2, Part 16.3).
- ☐ Services communicate by container/service name, never hardcoded IPs (Part 14.3).
- ☐ Only the ports that genuinely need external access are published; internal services stay unpublished (Part 18.1).
- ☐ TLS termination handled appropriately (Nginx/ALB), backend traffic isolated within the trusted internal network (Part 33.4).

42.5 41.5 Storage

- ☐ Stateful data lives in named volumes or external managed services, never solely in a container's writable layer (Part 13.4).
- ☐ Backup strategy exists for anything in a volume that constitutes real, non-reproducible data.
- ☐ Bind mounts used deliberately (dev convenience, specific host config), not as a substitute for proper volumes in production.

42.6 41.6 Logging & Monitoring

- ☐ Application logs to `stdout/stderr`, not to files inside the container (Part 24.1).
- ☐ Log driver configured with rotation limits (`max-size`/`max-file`) to prevent unbounded host disk growth (Part 24.2).
- ☐ Logs shipped to a centralized system in production, not relied on via `docker logs` alone (Part 24.4).
- ☐ Container and host metrics both monitored (`cAdvisor` + `node_exporter` → Prometheus → Grafana), with real alerting configured, not just dashboards nobody watches (Part 25.2).

42.7 41.7 CI/CD

- ☐ Tests run **before** the image is built/pushed (Part 34.3).
- ☐ Security scan is a genuine release gate, not advisory-only (Part 34.3).
- ☐ Images tagged with explicit, traceable versions (build number, git SHA) — never solely latest (Part 8.2, Part 36.4).
- ☐ Deployment step is separate and fast, distinct from infrastructure provisioning (Part 36.4).

42.8 41.8 Registry

- ☐ Registry access authenticated (IAM for ECR, tokens/credentials rotated appropriately) (Part 20.2).
- ☐ Old/unused image tags cleaned up on a lifecycle policy, rather than accumulating indefinitely.
- ☐ Registry itself access-controlled — not every CI job/developer needs push access to every repository.

42.9 41.9 Graceful Operation

- ☐ Application handles SIGTERM cleanly within the container-stop grace period (Part 12.5).
- ☐ Rolling deploys tested to confirm zero dropped requests under real traffic.
- ☐ Runbook/documentation exists for the troubleshooting scenarios in Part 26, specific to this application's known failure modes.

43 PART 42 — Hands-On Labs (50+)

Work through these in order on a real Ubuntu 24.04 host with Docker installed (Part 5). Each builds on skills from earlier labs.

43.1 Foundations (Labs 1-10)

1. **Install Docker** — complete Part 5's install steps, verify with `docker run hello-world`.
2. **Run Nginx** — `docker run -d -p 8080:80 nginx`, confirm it's reachable in a browser.
3. **Run an Ubuntu container interactively** — `docker run -it ubuntu bash`, explore, then exit and note the container is now stopped, not gone.
4. **Interactive container exploration** — start a container, use `docker exec -it` to open a second shell into the same running container, observe both sessions share the same process namespace.
5. **Build your first image** — write a one-line Dockerfile (`FROM alpine + CMD ["echo", "hello"]`), build and run it.
6. **Write a real Dockerfile** — containerize a “hello world” script in a language of your choice, following Part 9's instruction reference.
7. **Use environment variables** — modify your app to read a config value from an env var; run it with `-e` and confirm the behavior changes.
8. **Map ports** — run two containers of the same image on different host ports simultaneously.
9. **Create volumes** — create a named volume, write data into it from one container, recreate the container, confirm the data persists.
10. **Create a custom network** — `docker network create`, attach two containers, ping/curl between them by name.

43.2 Application Building (Labs 11-15)

11. **Connect two containers** — reproduce Lab 10 with a real client/server pair (e.g., a simple HTTP server + a curl-based client container).
12. **Build a Python application image** — containerize a small Flask app per Part 32.
13. **Build a Node.js application image** — containerize a small Express app, following the same principles.
14. **Multi-stage build** — convert a Go or Node build to a proper multi-stage Dockerfile (Part 11), compare image sizes before/after.
15. **Optimize an image** — take Lab 12 or 13's image, apply `.dockerignore`, layer reordering (Part 29), and a slimmer base image; measure the size reduction.

43.3 Compose & Multi-Container Apps (Labs 16-20)

16. **Docker Compose basics** — convert two related `docker run` commands into a single `docker-compose.yml`.
17. **Flask + PostgreSQL** — build the Part 18.1-style stack, confirm data persists across `docker compose down/up` (without `-v`).
18. **Nginx + Flask** — put Nginx in front of your Flask app as a reverse proxy (Part 33.2).
19. **Redis + Flask** — add a Redis cache/session store, confirm Flask can read/write to it by service name.
20. **Health checks** — add HEALTHCHECK/Compose healthcheck blocks to every service in Lab 17-19's stack, and a proper `depends_on: condition: service_healthy`.

43.4 Resources, Logging, Security (Labs 21-30)

21. **Resource limits** — apply `--cpus/--memory` to a container, then intentionally load it past the limit and observe the behavior (throttling for CPU, OOM kill for memory).
22. **Logging** — configure the `json-file` driver with `max-size/max-file`, generate enough log volume to observe rotation happening.
23. **Push to a registry** — create a free Docker Hub account, `docker login`, tag and push an image you built.
24. **Docker Hub practice** — pull an image you don't own, inspect its layers/history, note its base image and size.
25. **Push to ECR** — create an ECR repository (via AWS Console or Terraform), authenticate, push an image.
26. **Image scanning** — install Trivy, scan one of your own images plus a known-old public image, compare the CVE counts.
27. **Non-root container** — take an existing Dockerfile that runs as root, add a `USER` instruction, confirm the app still works and `id` inside the container shows a non-root UID.
28. **Read-only filesystem** — run a container with `--read-only --tmpfs /tmp`, confirm the app still functions (or diagnose what breaks and why).
29. **Docker security review** — audit one of your own images against Part 41.2's checklist and fix at least two gaps.
30. **Docker monitoring** — run `docker stats` while load-testing a container (e.g., with `ab` or `hey`), observe CPU/memory response.

43.5 Troubleshooting (Labs 31-35)

31. **Troubleshoot a container** — intentionally misconfigure an entrypoint so the container exits immediately; diagnose and fix using only `docker logs/docker inspect`.
32. **Troubleshoot networking** — put two containers on different networks, reproduce the “can't reach each other” failure, then fix it.
33. **Troubleshoot storage** — mount a volume at the wrong path, reproduce “data doesn't persist,” then fix it.
34. **Troubleshoot high CPU** — write a container that spins the CPU, identify it via `docker stats`, and apply a `--cpus` limit as a fix.
35. **Troubleshoot memory** — write a container with a deliberate memory leak, observe the OOM kill, set an appropriate `--memory` limit and alert threshold.

43.6 CI/CD & Infrastructure (Labs 36-40)

36. **Jenkins + Docker** — set up a local/test Jenkins instance, build a pipeline that builds and tags a Docker image on every commit (Part 34.2).
37. **Terraform + AWS + Docker** — use Terraform to provision an ECR repository and a Security Group for an EC2 instance (Part 36.3).
38. **EC2 + Docker** — launch that EC2 instance, install Docker via `user-data/cloud-init` (Linux course Part 27.4), pull and run an image from your ECR repo.
39. **ECR + ECS** — extend Lab 37's Terraform to provision an ECS Fargate service running your image, fronted by an ALB (Part 35.4).
40. **Docker + Kubernetes** — (using a local cluster like `kind` or `minikube`) deploy the same image as a Kubernetes Pod, and compare the YAML to your Compose file's structure.

43.7 Production Patterns (Labs 41-50+)

41. **Dockerized microservices** — build two small services that communicate over HTTP, each in its own container, on a shared custom network.

42. **Reverse proxy** — put Nginx in front of both microservices from Lab 41, routing by path (/service-a/, /service-b/).
43. **SSL termination** — add a self-signed certificate to your Lab 42 Nginx config and serve over HTTPS (Part 33.4).
44. **Containerized database with backups** — extend Lab 17's Postgres setup with a scheduled pg_dump backup script writing to a separate volume.
45. **Production deployment** — take Lab 39's ECS deployment and add: health checks, a proper restart/rolling-update strategy, and CloudWatch log shipping.
46. **Full CI/CD pipeline** — connect Lab 36's Jenkins pipeline all the way through to Lab 39's ECS service, so a git push results in an automatic, tested, scanned, deployed update.
47. **Monitoring with Prometheus** — deploy cAdvisor + Prometheus alongside your Lab 41 microservices (Part 25.2), scrape their container metrics.
48. **Grafana dashboard** — build a Grafana dashboard on top of Lab 47's Prometheus data, showing CPU/memory/request metrics for your services.
49. **Security scanning pipeline** — add Trivy scanning as a required, failing gate in Lab 46's pipeline (Part 34.3), and generate/store an SBOM per build (Part 40.3).
50. **Complete production DevOps project** — combine everything into Part 45's full capstone architecture: GitHub → Jenkins → Docker → Registry → Terraform → AWS → Kubernetes → Prometheus → Grafana, for a real multi-service application.
51. **(Extension) Image signing** — add Cosign signing to Lab 49's pipeline, and signature verification before deployment (Part 40.4).
52. **(Extension) Rootless Docker** — set up rootless Docker mode on a test host and compare the operational differences to standard Docker (Part 40.6).

44 PART 43 — Complete Docker Cheat Sheet

Format: **COMMAND → PURPOSE → IMPORTANT OPTIONS → EXAMPLE → DEVOPS USE CASE.**

44.1 Installation

- **docker version** → client+server version → — → docker version → confirm daemon reachable
- **docker info** → daemon config/status → — → docker info → check storage driver, cgroup version, root dir
- **systemctl status docker** → daemon service state → — → — → confirm the daemon is actually running

44.2 Images

- **docker images** → list local images → -a -q → docker images → audit what's on a host
- **docker pull** → download an image → IMAGE:TAG → docker pull nginx:1.27 → fetch a specific version
- **docker push** → upload an image → — → docker push myrepo/app:1.0 → publish to a registry
- **docker build** → build from a Dockerfile → -t -f --no-cache → docker build -t myapp:1.0 . → CI build step
- **docker image inspect** → full metadata → — → docker image inspect nginx → debug image config
- **docker image history** → per-layer breakdown → — → docker image history myapp → find what's bloating size
- **docker rmi** → delete an image → -f → **[DANGEROUS]** docker rmi myapp:old → cleanup

- **docker tag** → add a tag → — → `docker tag myapp:1.0 myapp:latest` → prep for registry push
- **docker save/load** → export/import as tar → `-o -i` → `docker save -o app.tar myapp` → offline transfer

44.3 Containers

- **docker run** → create+start → many (Part 7) → `docker run -d -p 8080:80 nginx` → launch a service
- **docker create/start** → create without starting / start existing → `-a` → `docker start app` → controlled startup
- **docker stop** → graceful stop (SIGTERM→SIGKILL) → `-t` → `docker stop app` → safe shutdown
- **docker restart** → stop+start → — → `docker restart app` → apply a config reload
- **docker kill** → immediate SIGKILL → `-s` → `docker kill app` → force-stop, no cleanup
- **docker rm** → delete a stopped container → `-f -v` → **[DANGEROUS]** `docker rm -f app` → cleanup
- **docker ps** → list containers → `-a -q` → `docker ps -a` → check status of everything
- **docker logs** → view stdout/stderr → `-f --tail --since` → `docker logs -f app` → live debugging
- **docker exec** → run a command inside a running container → `-it` → `docker exec -it app bash` → interactive debug
- **docker inspect** → full JSON state → `--format` → `docker inspect app` → check mounts/env/exit code
- **docker cp** → copy files host↔container → — → `docker cp app:/log.txt .` → pull a log/artifact out
- **docker stats** → live resource usage → `--no-stream` → `docker stats` → spot a resource hog
- **docker top** → processes inside a container → — → `docker top app` → check without needing exec

44.4 Dockerfile Instructions

- **FROM** → base image → — → `FROM python:3.12-slim`
- **RUN** → build-time command → — → `RUN pip install -r requirements.txt`
- **CMD** → default (overridable) startup command → exec form preferred → `CMD ["python3", "app.py"]`
- **ENTRYPOINT** → fixed startup command → exec form preferred → `ENTRYPOINT ["python3", "app.py"]`
- **COPY** → copy files in → — → `COPY . .`
- **WORKDIR** → set working directory → — → `WORKDIR /app`
- **ENV** → runtime-visible variable → — → `ENV NODE_ENV=production`
- **ARG** → build-time-only variable → — → `ARG VERSION=1.0`
- **EXPOSE** → document listening port (metadata only!) → — → `EXPOSE 8080`
- **USER** → run as this user → — → `USER appuser`
- **HEALTHCHECK** → define container health check → `--interval --retries` → `HEALTHCHECK CMD curl -f localhost/health`

44.5 Volumes

- **docker volume create** → make a named volume → — → `docker volume create pgdata`
- **docker volume ls** → list volumes → — → `docker volume ls`

- **docker volume inspect** → volume details/mountpoint → — → `docker volume inspect pgdata`
- **docker volume rm** → delete a volume → — → **[DANGEROUS]** `docker volume rm pgdata`
- **docker volume prune** → delete all unreferenced volumes → — → **[DANGEROUS]** `docker volume prune`

44.6 Networks

- **docker network create** → make a custom network → — → `docker network create app-net`
- **docker network ls** → list networks → — → `docker network ls`
- **docker network inspect** → see attached containers/config → — → `docker network inspect app-net`
- **docker network connect/disconnect** → attach/detach a running container → — → `docker network connect app-net app`

44.7 Compose

- **docker compose up** → start the whole stack → -d --build → `docker compose up -d`
- **docker compose down** → stop and remove → -v (**[DANGEROUS]**, deletes volumes) → `docker compose down`
- **docker compose ps** → project container status → — → `docker compose ps`
- **docker compose logs** → logs across services → -f → `docker compose logs -f backend`
- **docker compose exec** → run a command in a service → — → `docker compose exec backend bash`
- **docker compose config** → validate/print resolved config → — → `docker compose config`

44.8 Registries

- **docker login** → authenticate → -u --password-stdin → `docker login`
- **docker tag** → prep an image for a registry → — → `docker tag myapp registry/myapp:1.0`
- **docker push/pull** → upload/download → — → `docker push registry/myapp:1.0`
- **aws ecr get-login-password** → ECR auth token → \| `docker login --password-stdin` → CI/CD registry auth

44.9 Security

- **--cap-drop=ALL --cap-add=X** → least-privilege capabilities → — → `docker run --cap-drop=ALL myapp`
- **--read-only** → read-only root filesystem → --tmpfs for writable exceptions → `docker run --read-only myapp`
- **--user** → run as non-root → UID:GID → `docker run --user 1000:1000 myapp`
- **--privileged** → disable nearly all isolation → — → **[DANGEROUS]** avoid for app containers
- **trivy image** → vulnerability scan → --severity → `trivy image --severity HIGH,CRITICAL myapp`
- **cosign sign/verify** → image signing → --key → `cosign sign --key cosign.key myapp:1.0`

44.10 Monitoring

- **docker stats** → live CPU/mem/net/IO → --no-stream → `docker stats`

- **docker events** → live daemon event stream → `--filter` → `docker events --filter event=oom`
- **docker top** → process list inside a container → `—` → `docker top app`

44.11 Troubleshooting

- **docker logs** → app output → `-f --tail` → `docker logs -f app`
- **docker inspect** → full state, exit codes, mounts → `--format` → `docker inspect --format '{{.State.OOMKilled}}' app`
- **docker diff** → filesystem changes vs image → `—` → `docker diff app`
- **docker exec ... ps/ss/curl/nslookup** → Linux-level diagnostics inside a container → `—` → `docker exec app curl backend:5000`

44.12 System Cleanup

- **docker system df** → disk usage breakdown → `-v` → `docker system df -v`
- **docker system prune** → remove unused data → `-a --volumes ([DANGEROUS])` → `docker system prune -a`
- **docker container prune** → remove stopped containers only → `—` → `docker container prune`
- **docker image prune** → remove dangling (or all unused with `-a`) images → `-a` → `docker image prune -a`
- **docker volume prune** → remove unreferenced volumes → `—` → **[DANGEROUS]** `docker volume prune`
- **docker network prune** → remove unused networks → `—` → `docker network prune`

45 PART 44 — Docker & Container Interview Preparation

Each entry: **short answer**, then **detailed explanation**, then a **real-world example**.

45.1 Docker Fundamentals

Q: What is the difference between a container and a virtual machine? Short answer: a VM virtualizes hardware and runs its own full kernel; a container is an isolated Linux process sharing the host's kernel, using namespaces and cgroups. Detail: this is why containers start in milliseconds and are much smaller, but have a weaker isolation boundary than VMs. Example: "We moved a fleet of small internal tools from one-VM-per-app to containers on shared hosts, cutting our EC2 spend significantly through much higher density."

Q: What is the difference between an image and a container? Short answer: an image is a read-only template; a container is a running instance of that image with a writable layer on top. Detail: many containers can run from the same image simultaneously, each with its own independent writable layer. Example: "We run three replicas of the same myapp:1.4.2 image — one image, three independent containers."

45.2 Containers/Internals

Q: What Linux kernel features does Docker actually rely on? Short answer: namespaces (isolation), cgroups (resource limits), and OverlayFS (layered filesystem). Detail: Docker doesn't invent isolation — it packages existing kernel primitives with a great build/run/distribute experience. Example: "`docker inspect --format '{{.State.Pid}}'`" shows a container's PID on the host — proof it's just a normal Linux process underneath."

Q: Why does PID 1 matter inside a container? Short answer: PID 1 has special signal-handling and zombie-reaping responsibilities; a shell wrapper as PID 1 often fails to forward SIGTERM to the real app. Detail: use exec in entrypoint scripts, or a minimal init like tini, to ensure the real application receives signals correctly. Example: “Our shutdowns were taking the full 10-second SIGKILL timeout every time until we added exec before the app launch in our entrypoint script.”

45.3 Images

Q: Why shouldn't you rely on the latest tag in production? Short answer: latest is just a mutable, unpinned pointer — it can silently point to a different image over time, breaking reproducibility. Detail: use explicit semantic version tags, or digest pinning for maximum reproducibility. Example: “A latest-tagged base image update silently introduced a breaking dependency change; we now pin every base image to an explicit patch version.”

Q: Your image is 2GB. How do you reduce it? Short answer: switch to a smaller base image, use multi-stage builds, add a .dockerignore, and review docker image history for the biggest layers. Detail: multi-stage builds usually give the single biggest win — compiling with a full toolchain, shipping only the artifact. Example: “Switching a Go service from a full golang image to a multi-stage build with a scratch final stage took us from 900MB to under 20MB.”

45.4 Dockerfile

Q: CMD vs ENTRYPOINT — when do you use each? Short answer: ENTRYPOINT is the fixed command that always runs; CMD provides default, overridable arguments (or a fully-overridable default command if used alone). Detail: the common best practice is ENTRYPOINT for the binary/interpreter, CMD for its default arguments. Example: “ENTRYPOINT [\"python3\", \"app.py\"] + CMD [\"--port\", \"8080\"] lets docker run myimage --port 9090 change the port without touching the Dockerfile.”

Q: Why does changing one line in a Dockerfile sometimes rebuild everything after it? Short answer: Docker's build cache is invalidated for a layer, and every subsequent layer, the moment any earlier instruction's cache is busted. Detail: order instructions from least to most frequently changing — dependency installs before source code copies. Example: “Moving COPY requirements.txt . + RUN pip install before COPY . . cut our average build time from 3 minutes to 20 seconds.”

45.5 Networking

Q: Why can't two containers reach each other by name, and how do you fix it? Short answer: they're likely on the default bridge network (no built-in DNS) or on two different networks entirely; fix by creating and attaching both to a custom network. Detail: the default bridge network provides IP connectivity but no name-based DNS resolution — always use a user-defined network for multi-container apps. Example: “Two Compose services in different projects (and therefore different default networks) couldn't see each other until we put them on one explicit shared network.”

Q: What's the difference between EXPOSE and -p? Short answer: EXPOSE is documentation only, -p/--publish is what actually maps a host port to a container port. Detail: this trips up almost every beginner at least once. Example: “We had EXPOSE 8080 in the Dockerfile and assumed the app was reachable — it wasn't, until we added -p 8080:8080 at runtime.”

45.6 Storage

Q: Bind mount vs named volume — when do you use each? Short answer: bind mounts point at a specific host path (great for local dev/live reload); named volumes are Docker-managed, portable, backup-friendly persistent storage (the standard for production data). Detail: bind mounts couple you to a specific host filesystem layout; volumes don't. Example: "Local dev uses a bind mount of the source directory for hot-reload; production uses a named volume for the Postgres data directory."

45.7 Compose

Q: Why use depends_on with a healthcheck condition instead of plain depends_on? Short answer: plain depends_on only waits for the container to start, not for the actual service inside to be ready. Detail: a database container can be "running" before Postgres is actually accepting connections — healthcheck-gated depends_on waits for genuine readiness. Example: "Intermittent 'connection refused' errors on cold starts disappeared once we switched to condition: service_healthy for our database dependency."

45.8 Security

Q: Why is --privileged dangerous? Short answer: it disables nearly every container security mechanism at once — all capabilities, seccomp filtering, AppArmor/SELinux confinement, and grants direct host device access — making container escape realistically possible. Detail: legitimate use cases are narrow (certain low-level tooling); never use it for a normal application container. Example: "A well-known container-escape technique specifically requires --privileged or an equivalent overly-broad capability grant — removing it from our CI runners closed that path entirely."

Q: How do you avoid putting secrets in a Docker image? Short answer: never use ENV/ARG for real secrets; use file-mounted Docker secrets, or fetch from a secrets manager at runtime using an IAM role. Detail: anything baked into a layer is recoverable via docker history/docker inspect, even if a later layer tries to remove it. Example: "We found a database password in docker history on an old image months after it was 'removed' from the Dockerfile in a later commit — the original layer was still there."

45.9 Performance

Q: A container is consuming 100% CPU. What do you investigate? Short answer: docker stats to confirm and identify it, then docker top/exec ... top to find the specific process, then application-level profiling or strace to understand what it's actually doing. Detail: distinguish a genuine traffic-driven need to scale from a bug (infinite loop, pathological regex) — the fix is very different for each. Example: "docker stats isolated it to one container; docker exec ... top showed a single runaway worker process; turned out to be a regex with catastrophic backtracking on malicious input."

45.10 Troubleshooting (Scenario-Based)

Q: A container works locally but fails in production. How do you troubleshoot it? Short answer: compare the exact image (by digest, not just tag) running in both environments, compare environment variables/secrets, and check for environment-specific network/resource constraints. Detail: "works on my machine" for containers usually means it's actually a **different image or different config**, not truly the same container behaving differently — verify this first before looking elsewhere. Example: "Production was still on an older cached image

because the deploy step pulled `:latest` before the new tag had finished propagating — pinning to the exact CI-built tag fixed the class of bug entirely.”

Q: Two containers cannot communicate. What do you check? Short answer: are they on the same custom network (`docker network inspect`)? Are they using the correct container/service name? Is the target’s port actually listening (`docker exec ... ss -tlnp`)? Is there an application-level (not just network-level) reason for the refusal? Detail: work the chain systematically rather than guessing — this mirrors the Linux course’s networking troubleshooting chain, scoped to Docker’s networking layer. Example: “It looked like a networking bug, but `docker network inspect` showed both containers correctly attached — the real issue was the backend hadn’t finished starting yet and wasn’t listening on its port.”

Q: Your Docker image is 2GB. How do you reduce it? *(also covered under Images above — expect this exact framing in interviews)* Short answer: multi-stage build, smaller base image, `.dockerignore`, `docker image history` audit to find the biggest contributors. Example: as above.

45.11 CI/CD

Q: Where should security scanning happen in a CI/CD pipeline, and why? Short answer: after the build, before the push to a registry — as a genuine, failing gate, not an advisory-only report. Detail: scanning after push means a vulnerable image may already be pulled/deployed elsewhere before anyone reviews the scan result. Example: “We moved Trivy from a post-deploy nightly job to a pre-push CI gate with `--exit-code 1` on critical CVEs — it now blocks a small number of releases per month that would otherwise have reached production.”

45.12 AWS

Q: ECS vs EKS — how do you decide? Short answer: ECS is AWS-native, simpler to operate, tightly integrated with other AWS services; EKS is managed Kubernetes, more complex but portable and far more feature-rich/extensible. Detail: the underlying Docker image/build/registry workflow (Parts 4–20) is identical either way — the choice is purely about the orchestration layer. Example: “We started on ECS/Fargate for simplicity with a small team, and migrated to EKS once we needed Kubernetes-specific tooling (Helm charts, custom operators) our growing platform team wanted to standardize on.”

45.13 Kubernetes

Q: Does Kubernetes still use Docker? Short answer: not `dockerd` directly (removed via `dockershim`’s deprecation in 1.24) — but Kubernetes runs Docker-**built** images just fine, via `containerd` (the same component Docker itself uses) and `runc`. Detail: the image format (OCI) is standardized and vendor-neutral; only the specific daemon Kubernetes talks to changed. Example: “Our EKS clusters run `containerd` directly — but every image is still built with plain `docker build` in Jenkins, with zero changes needed when we migrated off `dockershim` years ago.”

45.14 Container Internals

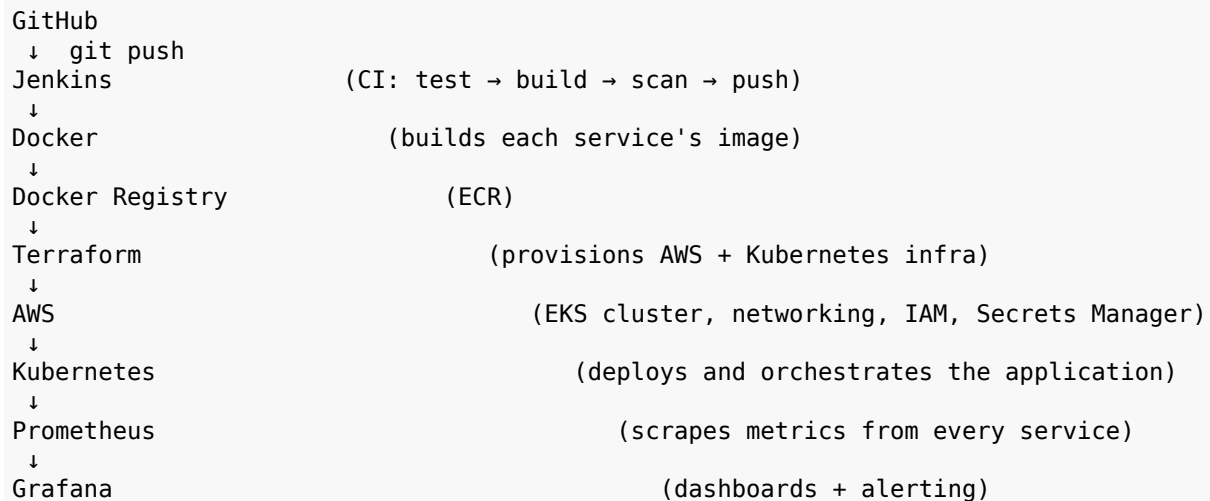
Q: What’s the difference between a namespace and a cgroup? Short answer: namespaces control **what a process can see** (isolation); cgroups control **how much it can use** (resource limits). Detail: both are required for “a container” as a concept — one without the other gives you either isolation with no limits, or limits with no isolation. Example: “A container with no `--memory` limit is still isolated (its own PID/network namespace) but can still exhaust host memory — namespaces alone don’t prevent that; cgroups do.”

46 PART 45 — Real DevOps Project: Full Production-Style Containerized Stack

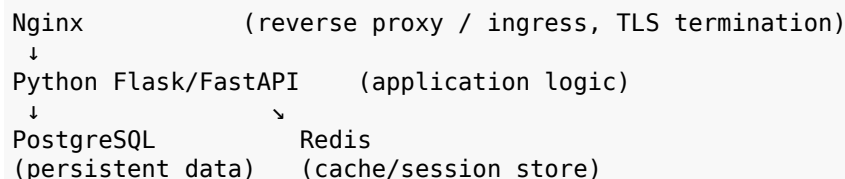
46.1 Project Goal

Build and ship a real multi-service application through the entire chain this course has covered — from a Dockerfile to a monitored, secured, Kubernetes-deployed production system.

46.2 Architecture



46.3 Application



46.4 Step-by-Step

1. Application and Dockerfiles - Flask/FastAPI service with a /health endpoint (Part 23.3), connecting to Postgres and Redis by service name (Part 14.3). - A production Dockerfile per Part 32.4: pinned base, non-root user, Gunicorn, health check. - Multi-stage build if any compiled dependencies are involved (Part 11).

2. Local Development with Compose

```
services:
  nginx:
    image: nginx:1.27
    ports: ["80:80"]
    volumes: [".:/nginx.conf:/etc/nginx/conf.d/default.conf:ro"]
    depends_on: [api]
    networks: [app-net]
```

```

api:
  build: ./api
  env_file: [.env.api]
  depends_on:
    database: {condition: service_healthy}
    redis: {condition: service_started}
  networks: [app-net]

redis:
  image: redis:7-alpine
  networks: [app-net]

database:
  image: postgres:16
  environment: {POSTGRES_DB: appdb}
  volumes: ["pgdata:/var/lib/postgresql/data"]
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U postgres"]
  networks: [app-net]

networks: {app-net: {}}
volumes: {pgdata: {}}

```

Use this stack for local development and integration testing — the exact same images (rebuilt via CI) go on to run in production, honoring Part 1.9's immutable-infrastructure principle end to end.

3. CI/CD Pipeline (Jenkins) - Checkout → run tests → docker build per service → Trivy scan (fail on critical/high, Part 21.8/34.3) → tag with the Git SHA/build number → push to ECR → trigger deployment.

4. Infrastructure as Code (Terraform) - VPC/subnets, an EKS cluster, ECR repositories per service, IAM roles (for CI, for the EKS node group, for Kubernetes service accounts via IRSA), and a Secrets Manager entry for the database password. - terraform plan → review → terraform apply.

5. Kubernetes Manifests

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: api
spec:
  replicas: 3
  selector:
    matchLabels: {app: api}
  template:
    metadata:
      labels: {app: api}
    spec:
      containers:
        - name: api
          image: <account>.dkr.ecr.us-east-1.amazonaws.com/api:GIT_SHA
          ports: [{containerPort: 8000}]
          resources:
            requests: {cpu: "250m", memory: "256Mi"}
            limits: {cpu: "500m", memory: "512Mi"}
          livenessProbe:
            httpGet: {path: /health, port: 8000}

```

```

        initialDelaySeconds: 10
      readinessProbe:
        httpGet: {path: /health, port: 8000}
        envFrom:
          - secretRef: {name: api-secrets}
    ---
  apiVersion: v1
  kind: Service
  metadata:
    name: api
  spec:
    selector: {app: api}
    ports: [{port: 8000, targetPort: 8000}]

```

Notice the direct mapping to concepts already covered: resources = --cpus/--memory (Part 22); livenessProbe/readinessProbe = HEALTHCHECK, split properly (Part 23.4); image referencing the ECR repository Terraform created and CI pushed to (Parts 20, 36); Service providing name-based discovery, the Kubernetes-scale equivalent of Docker's embedded DNS (Part 16).

6. Monitoring - Deploy kube-prometheus-stack (or equivalent) via Helm — Prometheus scrapes each service's metrics endpoint plus node/cAdvisor-equivalent metrics automatically in Kubernetes. - Build a Grafana dashboard covering request rate, error rate, latency (the "RED" method), plus CPU/memory per pod. - Alert on: elevated error rate, pod restart count, memory approaching limit, and the standard infra signals from the Linux course's troubleshooting checklist.

7. Security - Non-root containers, dropped capabilities, read-only root filesystems where feasible (Part 21). - Secrets sourced from Kubernetes Secrets backed by AWS Secrets Manager (via a controller like External Secrets Operator), never baked into images (Part 19). - Images scanned and signed in CI (Parts 21.8, 40.4).

8. Verify End to End

```

kubectl get pods
kubectl logs -f deployment/api
curl -I https://your-domain.com/health

```

This project deliberately exercises nearly every part of this course: Dockerfiles and multi-stage builds (Parts 9–11), storage and networking (Parts 13–16), Compose for local dev (Part 17), registries (Part 20), security (Parts 21, 40), resource limits and health checks (Parts 22–23), logging and monitoring (Parts 24–25), CI/CD (Part 34), AWS (Part 35), Terraform (Part 36), and Kubernetes (Part 37).

47 Complete Docker Roadmap

1. **Week 1** — Container fundamentals, history, Linux technologies behind containers (Parts 1–3).
2. **Week 2** — Docker architecture, installation, full CLI, docker run (Parts 4–7).
3. **Week 3** — Images and Dockerfiles, including multi-stage builds and best practices (Parts 8–11).
4. **Week 4** — Lifecycle, storage, networking, DNS (Parts 12–16).
5. **Week 5** — Compose, multi-container apps, secrets (Parts 17–19).
6. **Week 6** — Registries, security, resource management, health checks, logging, monitoring (Parts 20–25).

7. **Week 7** — Troubleshooting, debugging, performance, build cache, BuildKit, image optimization (Parts 26–31).
8. **Week 8** — Real application patterns, CI/CD, AWS, Terraform (Parts 32–36).
9. **Week 9** — Kubernetes relationship, Swarm, OCI, supply-chain security, production checklist (Parts 37–41).
10. **Week 10+** — Labs, capstone project, ongoing practice (Parts 42, 45), revisited continuously.

48 30-Day Docker Learning Plan

Days	Focus	Deliverable
1–2	Parts 1–3: fundamentals, history, Linux internals	Can explain namespaces/cgroups/OverlayFS from memory
3–4	Parts 4–6: architecture, install, CLI	Docker installed, comfortable with the full container/image CLI
5	Part 7: docker run deep dive	Can build the fully-worked example command from memory
6–8	Parts 8–11: images, Dockerfiles, best practices, multi-stage	Labs 5, 6, 14, 15 done
9–10	Parts 12–13: lifecycle, storage	Labs 9, 17 done; understands PID 1/signals
11–12	Parts 14–16: networking, ports, DNS	Labs 10, 11 done
13–15	Parts 17–19: Compose, multi-container apps, secrets	Labs 16–19 done
16	Part 20: registries	Lab 23, 25 done
17–18	Part 21: security	Labs 27–29 done
19	Parts 22–23: resources, health checks	Lab 21 done
20	Parts 24–25: logging, monitoring	Lab 22, 30 done
21–22	Parts 26–27: troubleshooting, debugging	Labs 31–35 done
23	Parts 28–31: performance, cache, BuildKit, base images	Rebuilt an image applying every optimization learned
24	Parts 32–33: Python, Nginx patterns	—
25–26	Part 34: CI/CD	Lab 36 done
27	Parts 35–36: AWS, Terraform	Labs 37–39 done
28	Part 37: Kubernetes relationship	Lab 40 done
29	Parts 38–40: Swarm, OCI, supply chain	Re-read all interview Qs (Part 44), could answer unaided
30	Part 45: capstone project	At least the Compose + CI + ECR portion running end to end

48.1 Daily Practice Schedule (~1.5-2 hrs/day)

- 30 min — read/review a section of this course.
 - 45 min — hands-on labs (Part 42), typing every command yourself.
 - 15 min — review yesterday's commands without notes (active recall).
 - 15 min — one troubleshooting scenario (Part 26), talked through out loud as if in an interview.
-

49 Docker Troubleshooting Checklist (Print/Keep Handy)

1. **Is the container actually running?** `docker ps -a`, check status and exit code.
 2. **What does it say?** `docker logs <container>` — always the first real diagnostic step.
 3. **What's its full state?** `docker inspect <container>` — mounts, env, restart policy, OOM status.
 4. **Networking** — `docker network inspect`, container/service names (not IPs), `docker exec ... curl/nslookup`.
 5. **Storage** — correct volume, correct mount path, `docker inspect` Mounts section.
 6. **Resources** — `docker stats`, check against configured `--cpus/--memory` limits.
 7. **Permissions** — `docker exec ... id, ls -l`, UID alignment for bind mounts.
 8. **Image freshness** — confirm you're actually running the image/tag/digest you think you are.
 9. **Isolate the layer** — Docker-level issue vs Linux-level issue vs application-level bug (Part 27.2).
 10. **Document the root cause and the fix** — feed it back into monitoring/alerting/the Part 41 checklist.
-

50 Quick-Reference: Course Connections

- **Docker + Linux** → Parts 1-3 (namespaces, cgroups, OverlayFS are the shared foundation with the Linux Mastery course).
 - **Docker + Jenkins** → Part 34 (test → build → scan → push → deploy pipeline).
 - **Docker + Terraform** → Part 36 (Docker builds the app; Terraform builds the infrastructure).
 - **Docker + AWS** → Part 35 (EC2, ECR, ECS/Fargate, ALB, CloudWatch, IAM, Secrets Manager).
 - **Docker + Kubernetes** → Part 37 (OCI images flow unchanged from `docker build` straight into Pods, via `containerd/runc`).
 - **Docker + Prometheus/Grafana** → Part 25.2 (`cAdvisor` + `node_exporter` → Prometheus → Grafana).
-

End of course. This document is meant to be a living reference — re-read Part 26 and re-run Part 42's labs periodically; real fluency comes from repetition against real (or realistically broken) systems, not from a single read-through.